

Devoir 1 - Économétrie des séries temporelles

“Lijin GAO (11364985), Steve Galarneau (98118040)”

Instructions

- Formez une équipe de trois membres au maximum pour réaliser le devoir.
- Fournissez un document PDF détaillé exporté à partir d'un fichier R Markdown (comme celui-ci), d'un notebook Python ou d'un script MATLAB. Le fichier doit contenir tout le code utilisé pour produire les résultats et répondre aux questions. Le code doit être clair et facile à comprendre, les figures doivent avoir des titres, les axes doivent être étiquetés, et les graphiques doivent comporter une légende. Des explications, justifications et interprétations sont obligatoires pour chaque question.
- Le devoir doit être soumis électroniquement sur ZoneCours (Remise de travaux).
- Toute forme de plagiat est inacceptable.
- Téléchargez le fichier de données disponible sur ZoneCours. Il contient des données mensuelles historiques sur le taux de change CAD/USD.

Question 1 - Stationnarité

1.a)

Chargez les données. Produisez un graphique de la série $z_t \equiv \ln(CADUSD_t)$ avec des étiquettes sur chaque axe et un titre $\ln(CADUSD_t)$. L'axe des abscisses doit représenter le temps.

```
#Clear the workspace
rm(list=ls())

#load required packages
library(readxl)

# Import the data (Excel file in the same folder as the Rmd file)
data <- read_excel("EXCAUS.xlsx", sheet = "Monthly")
head(data,10)
```

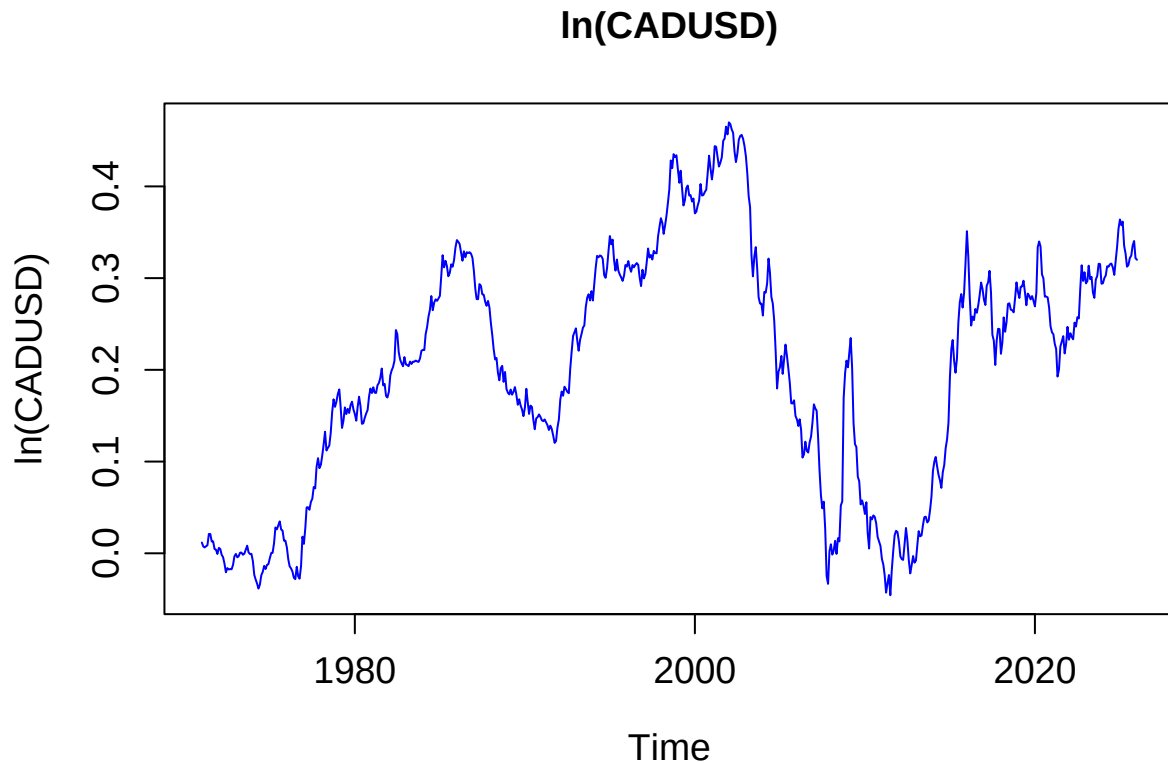
```
## # A tibble: 10 x 2
##   observation_date    EXCAUS
##   <dtm>              <dbl>
## 1 1971-01-01 00:00:00    1.01
```

```
## 2 1971-02-01 00:00:00 1.01
## 3 1971-03-01 00:00:00 1.01
## 4 1971-04-01 00:00:00 1.01
## 5 1971-05-01 00:00:00 1.01
## 6 1971-06-01 00:00:00 1.02
## 7 1971-07-01 00:00:00 1.02
## 8 1971-08-01 00:00:00 1.01
## 9 1971-09-01 00:00:00 1.01
## 10 1971-10-01 00:00:00 1.00
```

```
# Extract time and exchange rate
date <- as.Date(data$observation_date)
CADUSD <- data$EXCAUS

# Log-transform the exchange rate
z_t <- log(CADUSD)

# Plot the time series
plot(
  date, z_t,
  type = "l",
  col="blue",
  main = "ln(CADUSD)",
  xlab = "Time",
  ylab = "ln(CADUSD)",
  cex.axis = 1.2,
  cex.lab = 1.2,
  cex.main = 1.2
)
```



Interpretation: La série $\ln(\text{CADUSD}_t)$ ne semble pas stationnaire, car elle ne fluctue pas autour d'une moyenne constante et présente des variations persistantes au cours du temps.

1.b)

Supposons que la série z_t a une tendance quadratique déterministe : $\tau_t = \alpha + \beta t + \gamma t^2$. Appliquez la transformation nécessaire pour stationnariser z_t , puis faites un graphique de la série obtenue.

```
# Length of the series
T <- length(z_t)

# Time index t = 1,...,T
trend <- 1:T

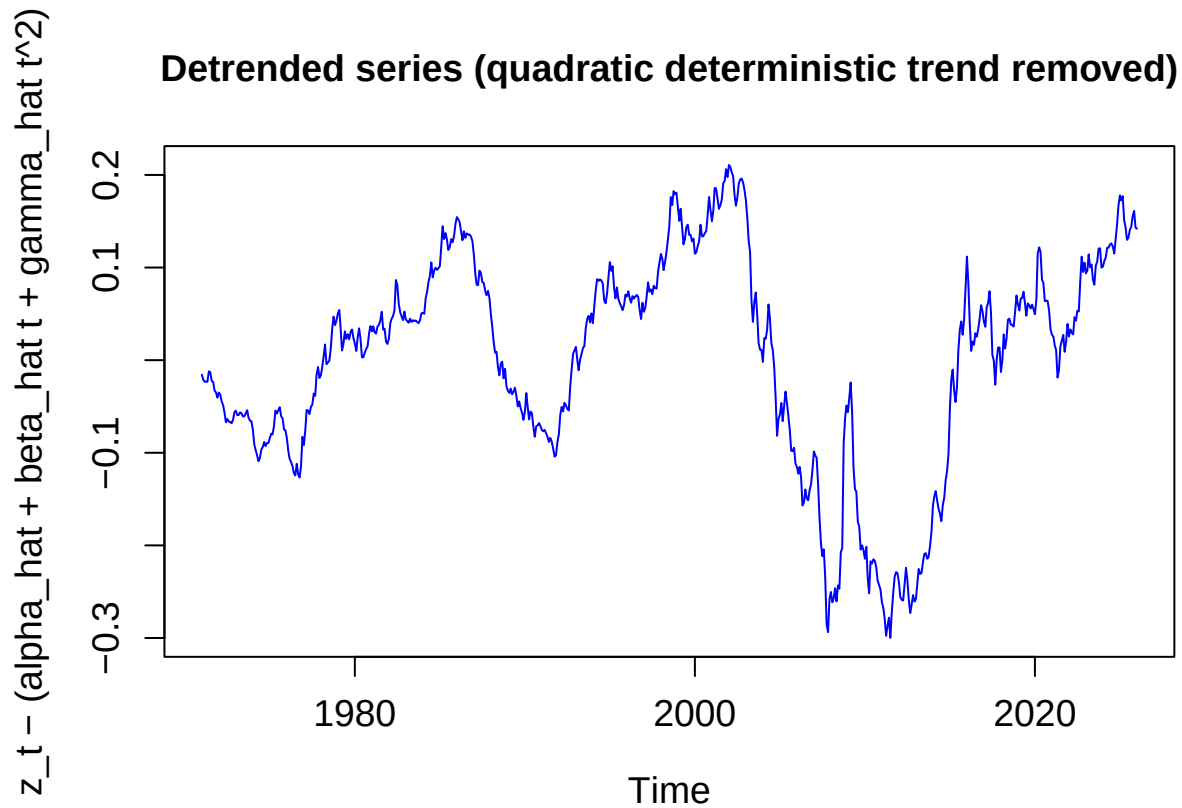
# Build regressors: intercept + t + t^2
X <- cbind(1, trend, trend^2)

# Dependent variable
Y <- z_t

# OLS estimator: B = (X'X)^(-1) X'Y (same formula as in the file)
B <- solve(crossprod(X), crossprod(X, Y))

# Detrended series (residuals): z_t - fitted trend
z_detrended <- Y - X %*% B
```

```
# Plot detrended series
plot(
  date, z_detrended,
  type = "l",
  col = "blue",
  main = "Detrended series (quadratic deterministic trend removed)",
  xlab = "Time",
  ylab = "z_t - (alpha_hat + beta_hat t + gamma_hat t^2)",
  cex.axis = 1.2,
  cex.lab = 1.2,
  cex.main = 1.2
)
```



Interpretation: Nous estimons que la série z_t suit une tendance déterministe quadratique de la forme $\alpha + \beta t + \gamma t^2$. Nous estimons cette tendance par la méthode des moindres carrés ordinaires (MCO) et la retirons de la série initiale. La série détrendée fluctue autour de zéro, ce qui indique que la composante déterministe quadratique a été éliminée. Toutefois, on observe encore une certaine persistance dans la dynamique de la série, ce qui suggère que la suppression d'une tendance déterministe pourrait ne pas suffire pour assurer la stationnarité.

1.c)

Supposons que la série z_t a une tendance stochastique. Appliquez la transformation requise pour stationnariser z_t , puis faites un graphique de la série obtenue.

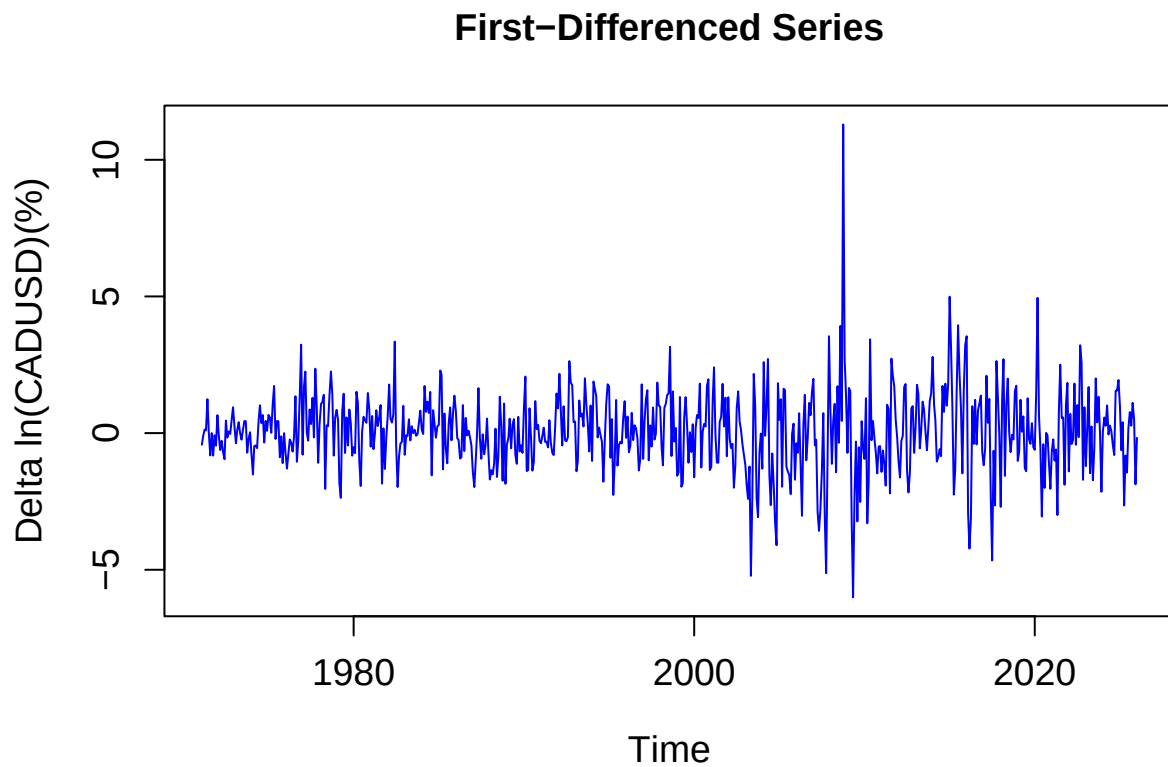
```

# First difference of the log series (growth rate in %)
dz <- diff(z_t)* 100

# Align dates (diff() reduces the length by 1)
date_dz <- date[-1]

# Plot the differenced series
plot(
  date_dz, dz,
  col="blue",
  type = "l",
  main = "First-Differenced Series",
  xlab = "Time",
  ylab = "Delta ln(CADUSD) (%)",
  cex.axis = 1.2,
  cex.lab = 1.2,
  cex.main = 1.2)

```

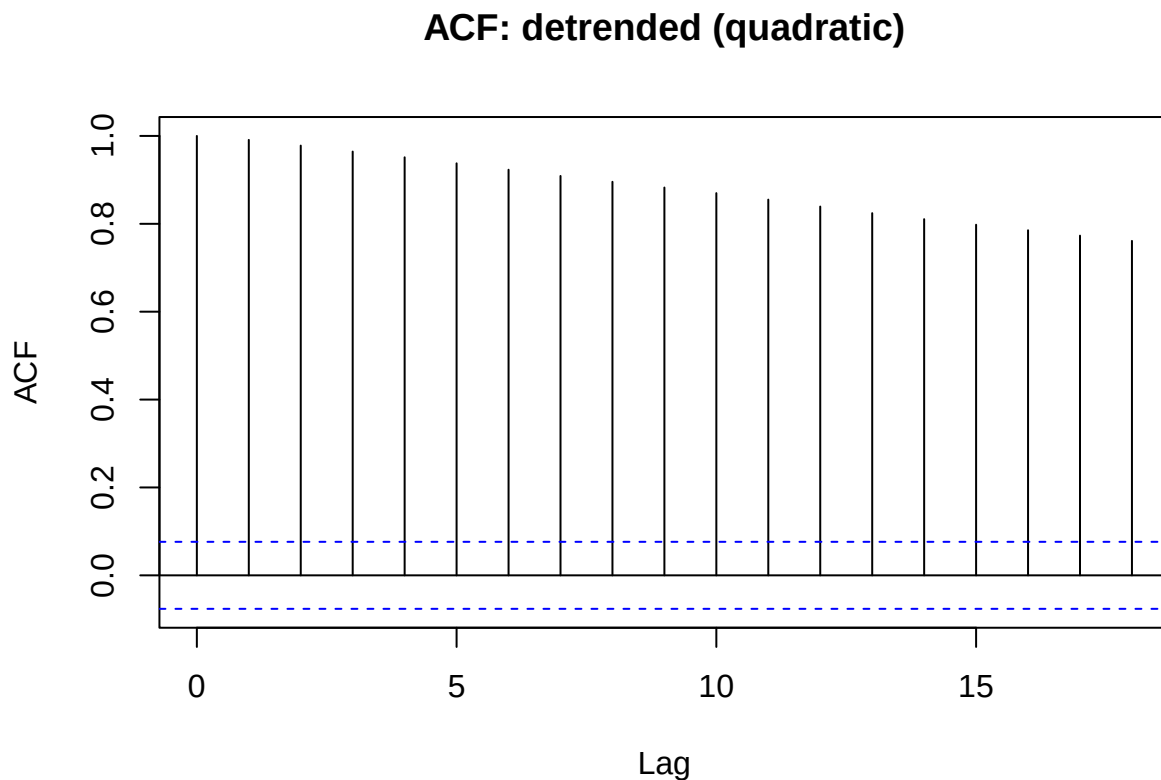


Interpretation: La première différence de $z_t = \ln(CADUSD)$ est utilisée afin de stationnariser la série en présence d'une tendance stochastique. La série obtenue fluctue autour de zéro sans tendance apparente, ce qui indique qu'elle est stationnaire. La moyenne semble constante bien qu'il y a parfois quelques fortes variations.

1.d)

Tracez et analysez les fonctions d'autocorrélation des échantillons des séries obtenues aux parties 1.b) et 1.c) afin de déterminer si les séries sont stationnaires. Laquelle des deux transformations produit une série stationnaire et pourquoi ?

```
#Assume for one and half year:lag of 18 months
acf1<-acf(z_detrended, lag.max = 18,
  main = "ACF: detrended (quadratic)",
  xlab = "Lag", ylab = "ACF")
```

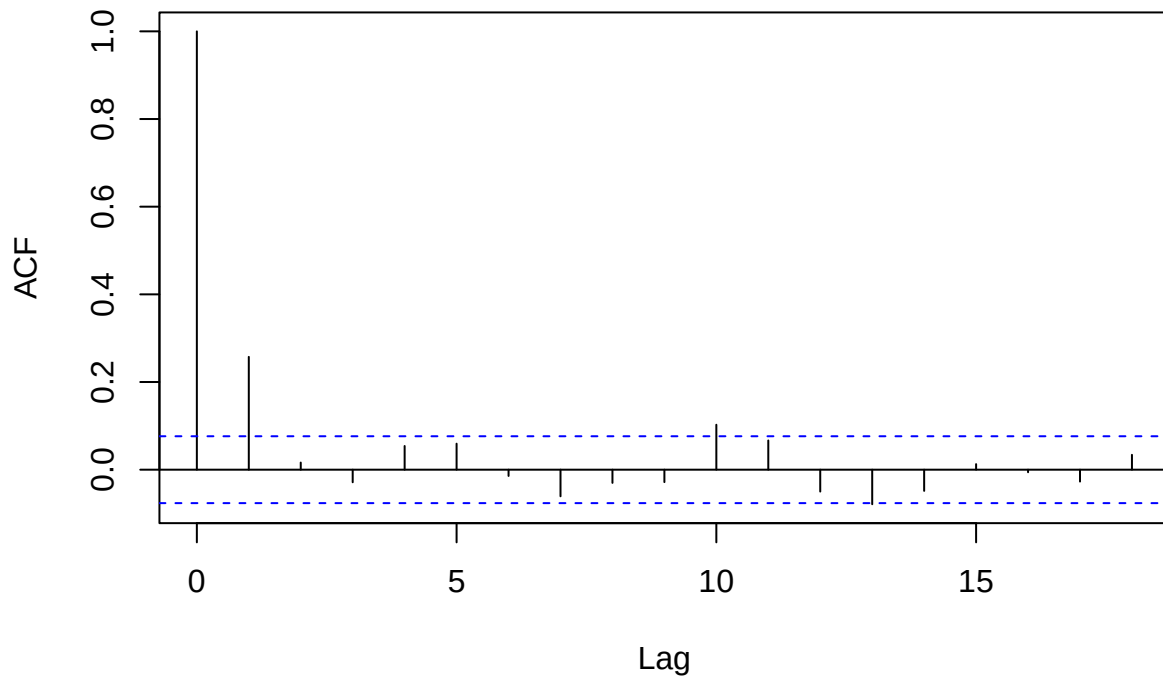


```
acf1
```

```
##
## Autocorrelations of series 'z_detrended', by lag
##
##      0      1      2      3      4      5      6      7      8      9     10     11     12
## 1.000 0.991 0.978 0.964 0.951 0.938 0.923 0.909 0.896 0.883 0.870 0.855 0.839
##     13     14     15     16     17     18
## 0.824 0.811 0.798 0.785 0.773 0.761
```

```
acf2<-acf(dz, lag.max = 18,
  main = "ACF: first difference",
  xlab = "Lag", ylab = "ACF")
```

ACF: first difference



```
acf2
```

```
##
## Autocorrelations of series 'dz', by lag
##
##      0      1      2      3      4      5      6      7      8      9     10
## 1.000 0.257 0.016 -0.029 0.054 0.059 -0.014 -0.061 -0.030 -0.028 0.103
##     11     12     13     14     15     16     17     18
## 0.067 -0.050 -0.079 -0.048 0.013 -0.006 -0.027 0.034
```

Interpretation: L'ACF de la série détrendée montre une décroissance très lente et des autocorrélations élevées, ce qui suggère la présence d'une forte persistance et donc une non-stationnarité. En revanche, l'ACF de la série en première différence décroît rapidement et la plupart des autocorrélations sont non significatives. Nous estimons que la transformation stochastique par première différence nous permet d'obtenir une série stationnaire.

Question 2 - Sélection des modèles

2.a)

À partir d'ici on va appeler la série stationnarisé y_t . Estimez les 8 modèles suivants par maximum de vraisemblance sur la série y_t , rapportez les résultats d'estimation et vérifiez si les conditions de stationnarité sont satisfaites.

$$\text{AR}(1) \ y_t = \delta + \phi_1 y_{t-1} + u_t$$

$$\text{AR}(2) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + u_t$$

$$\text{AR}(3) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + u_t$$

$$\text{AR}(4) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + \phi_4 y_{t-4} + u_t$$

$$\text{ARMA}(1,1) \ y_t = \delta + \phi_1 y_{t-1} + u_t - \theta_1 u_{t-1}$$

$$\text{ARMA}(2,2) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + u_t - \theta_1 u_{t-1} - \theta_2 u_{t-2}$$

$$\text{ARMA}(3,3) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + u_t - \theta_1 u_{t-1} - \theta_2 u_{t-2} - \theta_3 u_{t-3}$$

$$\text{ARMA}(4,4) \ y_t = \delta + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + \phi_4 y_{t-4} + u_t - \theta_1 u_{t-1} - \theta_2 u_{t-2} - \theta_3 u_{t-3} - \theta_4 u_{t-4}$$

```
library(forecast)
library(stats)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
# Série stationnarisée y_t
y_t <- as.numeric(dz)
y_t <- y_t[!is.na(y_t)]

# Estimer les 8 modèles (MLE)
AR1 <- forecast::Arima(y_t, order = c(1,0,0), include.mean = TRUE, method = "ML")
AR2 <- forecast::Arima(y_t, order = c(2,0,0), include.mean = TRUE, method = "ML")
AR3 <- forecast::Arima(y_t, order = c(3,0,0), include.mean = TRUE, method = "ML")
AR4 <- forecast::Arima(y_t, order = c(4,0,0), include.mean = TRUE, method = "ML")

ARMA11 <- forecast::Arima(y_t, order = c(1,0,1), include.mean = TRUE, method = "ML")
ARMA22 <- forecast::Arima(y_t, order = c(2,0,2), include.mean = TRUE, method = "ML")
ARMA33 <- forecast::Arima(y_t, order = c(3,0,3), include.mean = TRUE, method = "ML")
ARMA44 <- forecast::Arima(y_t, order = c(4,0,4), include.mean = TRUE, method = "ML")

# Liste NOMMÉE des modèles (important pour models AR1 etc.)
models <- list(
  AR1 = AR1, AR2 = AR2, AR3 = AR3, AR4 = AR4,
  ARMA11 = ARMA11, ARMA22 = ARMA22, ARMA33 = ARMA33, ARMA44 = ARMA44
)

model_labels <- c(
  AR1="AR(1)", AR2="AR(2)", AR3="AR(3)", AR4="AR(4)",
  ARMA11="ARMA(1,1)", ARMA22="ARMA(2,2)", ARMA33="ARMA(3,3)", ARMA44="ARMA(4,4)"
)
```



```

# Afficher les résultats
for (nm in names(models)) {
  cat(model_labels[nm], "\n")
  print(models[[nm]])
}

```

```

## AR(1)
## Series: y_t
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##          ar1      mean
##          0.2571  0.0466
## s.e.    0.0376  0.0731
##
## sigma^2 = 1.956: log likelihood = -1156.95
## AIC=2319.9  AICc=2319.94  BIC=2333.38
## AR(2)
## Series: y_t
## ARIMA(2,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2      mean
##          0.2708 -0.0534  0.0467
## s.e.    0.0388   0.0389  0.0693
##
## sigma^2 = 1.954: log likelihood = -1156.01
## AIC=2320.02  AICc=2320.08  BIC=2337.99
## AR(3)
## Series: y_t
## ARIMA(3,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2      ar3      mean
##          0.2697 -0.0477 -0.0208  0.0467
## s.e.    0.0389   0.0403   0.0389  0.0679
##
## sigma^2 = 1.956: log likelihood = -1155.87
## AIC=2321.73  AICc=2321.83  BIC=2344.19
## AR(4)
## Series: y_t
## ARIMA(4,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2      ar3      ar4      mean
##          0.2712 -0.0443 -0.0406  0.0730  0.0466
## s.e.    0.0388   0.0402   0.0402  0.0388  0.0730
##
## sigma^2 = 1.948: log likelihood = -1154.1
## AIC=2320.2  AICc=2320.33  BIC=2347.16
## ARMA(1,1)
## Series: y_t
## ARIMA(1,0,1) with non-zero mean

```

```
##
## Coefficients:
##      ar1      ma1      mean
##      0.1125  0.1562  0.0464
## s.e.  0.1251  0.1227  0.0707
##
## sigma^2 = 1.955: log likelihood = -1156.21
## AIC=2320.42  AICc=2320.48  BIC=2338.39
## ARMA(2,2)
## Series: y_t
## ARIMA(2,0,2) with non-zero mean
##
## Coefficients:
##      ar1      ar2      ma1      ma2      mean
##      -0.1894 -0.2786  0.4560  0.3540  0.0468
## s.e.   0.2762   0.1567  0.2702  0.1662  0.0668
##
## sigma^2 = 1.951: log likelihood = -1154.57
## AIC=2321.13  AICc=2321.26  BIC=2348.09
## ARMA(3,3)
## Series: y_t
## ARIMA(3,0,3) with non-zero mean
##
## Coefficients:
##      ar1      ar2      ar3      ma1      ma2      ma3      mean
##      0.2815 -0.3352  0.2782 -0.0127  0.3000 -0.2509  0.0470
## s.e.  0.9132   0.3404  0.1714   0.9073  0.5654   0.2374  0.0722
##
## sigma^2 = 1.952: log likelihood = -1153.79
## AIC=2323.58  AICc=2323.8   BIC=2359.52
## ARMA(4,4)
## Series: y_t
## ARIMA(4,0,4) with non-zero mean
##
## Coefficients:
##      ar1      ar2      ar3      ar4      ma1      ma2      ma3      ma4      mean
##      0.2867 -0.1960 -0.7063  0.1790 -0.0247  0.1546  0.7430  0.0878  0.0458
## s.e.  0.2207   0.2404   0.2425  0.1362   0.2216  0.1980  0.1993  0.1257  0.0732
##
## sigma^2 = 1.929: log likelihood = -1148.99
## AIC=2317.98  AICc=2318.32  BIC=2362.9
```

```
# ===== Vérification stationnarité (AR-part only) via matrice compagnon (comme Code 3) =====
#An AR(p) model is covariance-stationary iff all eigenvalues of the companion matrix lie strictly inside the unit circle
cat("\n--- Stationarity check (AR part): eigenvalue moduli < 1 ---\n")
```

```
##
## --- Stationarity check (AR part): eigenvalue moduli < 1 ---
```

```
for (nm in names(models)) {

  # Extract fitted model
  fit <- models[[nm]]
```

```

# Extract AR coefficients (phi_1, ..., phi_p)
ar_coefs <- fit$coef[grep("^ar", names(fit$coef))]

# Determine AR order p
p <- length(ar_coefs)

# -----
# Case 1: No AR terms (pure MA model)
# Pure MA models are always covariance-stationary
# -----
if (p == 0) {
  cat(model_labels[nm],
      "-> No AR terms (pure MA): covariance-stationary\n")
  next
}

# -----
# Case 2: AR(1)
# Companion matrix reduces to the scalar phi_1
# -----
if (p == 1) {
  m_phi <- matrix(ar_coefs, nrow = 1, ncol = 1)
}

# -----
# Case 3: AR(p), p >= 2
# Build the companion matrix:
else {
  m_phi <- rbind(
    matrix(ar_coefs, nrow = 1),
    cbind(diag(p - 1), rep(0, p - 1))
  )
}

# Compute eigenvalues of the companion matrix
eigvals <- eigen(m_phi, only.values = TRUE)$values
# Compute moduli (absolute values) of eigenvalues
moduli <- Mod(eigvals)

cat(model_labels[nm],
    "-> moduli:", paste(round(moduli, 4), collapse = ", "),
    "| Stationary:", all(moduli < 1), "\n")
}

## AR(1) -> moduli: 0.2571 | Stationary: TRUE
## AR(2) -> moduli: 0.2311, 0.2311 | Stationary: TRUE
## AR(3) -> moduli: 0.35, 0.35, 0.1701 | Stationary: TRUE
## AR(4) -> moduli: 0.5358, 0.5358, 0.531, 0.4787 | Stationary: TRUE
## ARMA(1,1) -> moduli: 0.1125 | Stationary: TRUE
## ARMA(2,2) -> moduli: 0.5278, 0.5278 | Stationary: TRUE
## ARMA(3,3) -> moduli: 0.7028, 0.7028, 0.5632 | Stationary: TRUE
## ARMA(4,4) -> moduli: 0.9527, 0.9527, 0.8267, 0.2386 | Stationary: TRUE

```

Interpretation: Les huit modèles estimés satisfont la condition de stationnarité : les modules des valeurs propres sont strictement inférieurs à 1. La série peut donc être considérée comme

covariance-stationnaire sous ces spécifications.

2.b)

Utilisez des tests statistiques pour la sélection de retards, des critères d'information et des tests d'autocorrélation des résidus (jusqu'à 18 retards) pour choisir deux modèles. Choisissez un modèle parmi AR(1), AR(2), AR(3), AR(4), et un autre parmi ARMA(1,1), ARMA(2,2), ARMA(3,3), ARMA(4,4).

```
# PART A - AR Model Selection (AR(1) to AR(4))
# Methods used:
# (1) Wald test:      H0:  $\phi_p = 0$  (last added AR lag)
# (2) Likelihood Ratio: AR(p) vs AR(p-1) (nested models)
# (3) Information criteria: AIC / BIC
# (4) Ljung-Box test: residual autocorrelation up to 18 lags

library(MASS)
library(forecast)
# (0) Settings

pbar <- 4          # Maximum AR order to consider (AR(1) to AR(4))
Ts <- length(y_t)  # Sample size
max_lag <- 18      # Maximum lag for Ljung-Box tests

# (1) Storage objects
# Matrices to store Wald, LR, and Information Criteria results

W <- matrix(NA, nrow = pbar, ncol = 4)  # Wald test results
LR <- matrix(NA, nrow = pbar, ncol = 4)  # Likelihood Ratio test results
AIC_mat <- matrix(NA, nrow = pbar, ncol = 2) # AIC values
BIC_mat <- matrix(NA, nrow = pbar, ncol = 2) # BIC values

# Assign column names
colnames(W) <- colnames(LR) <- c("lag", "test-stat", "crit-val", "p-value")
colnames(AIC_mat) <- colnames(BIC_mat) <- c("lag", "Info-Crit")

# 5% critical value from Chi-square(1) distribution
chi2_crit_5 <- qchisq(0.95, df = 1)

# (2) Wald tests + Likelihood Ratio tests + AIC/BIC
# Loop over AR(1) to AR(4)

for (p in 1:pbar) {
  # Restricted model (AR(p-1)) or white noise if p = 1
  if (p == 1) {
    fit0 <- arima(y_t, order = c(0, 0, 0), include.mean = TRUE)
  } else {
    fit0 <- arima(y_t, order = c(p - 1, 0, 0), include.mean = TRUE)
  }
  # Unrestricted model: AR(p)
  fit1 <- arima(y_t, order = c(p, 0, 0), include.mean = TRUE)

  # Wald Test
```

```

# H0: phi_p = 0 (last AR coefficient equals zero)

coefs <- fit1$coef           # Estimated coefficients
Vcov  <- fit1$var.coef       # Variance-covariance matrix
phi_name <- paste0("ar", p)  # Name of AR(p) coefficient

# Extract coefficient and its standard error
if (phi_name %in% names(coefs)) {
  phi_p <- coefs[phi_name]
  se_phi <- sqrt(Vcov[phi_name, phi_name])
} else {
  # Fallback in case naming differs
  phi_p <- coefs[p]
  se_phi <- sqrt(diag(Vcov))[p]
}

# Wald statistic ~ Chi-square(1)
W_val <- as.numeric((phi_p / se_phi)^2)

W[p, ] <- c(
  p,
  W_val,
  chi2_crit_5,
  1 - pchisq(W_val, df = 1)
)

# Likelihood Ratio Test
# H0: AR(p-1) is sufficient (no need for AR(p))
LR_val <- 2 * (as.numeric(logLik(fit1)) - as.numeric(logLik(fit0)))

LR[p, ] <- c(
  p,
  LR_val,
  chi2_crit_5,
  1 - pchisq(LR_val, df = 1)
)

# (3) Information Criteria
# Store AIC and BIC for model comparison
AIC_mat[p, ] <- c(p, AIC(fit1))
BIC_mat[p, ] <- c(p, BIC(fit1))
}
cat("\n--- Wald Test (H0: phi_p = 0) ---\n")

```

```

##
## --- Wald Test (H0: phi_p = 0) ---

```

```

print(round(W, 3))

```

```

##      lag test-stat crit-val p-value
## [1,] 1      46.798    3.841  0.000

```

```
## [2,] 2 1.887 3.841 0.170
## [3,] 3 0.287 3.841 0.592
## [4,] 4 3.540 3.841 0.060
```

```
cat("\n--- Likelihood Ratio Test (AR(p) vs AR(p-1)) ---\n")
```

```
##
## --- Likelihood Ratio Test (AR(p) vs AR(p-1)) ---
```

```
print(round(LR, 3))
```

```
##      lag test-stat crit-val p-value
## [1,] 1 45.180 3.841 0.000
## [2,] 2 1.885 3.841 0.170
## [3,] 3 0.287 3.841 0.592
## [4,] 4 3.529 3.841 0.060
```

```
cat("\n--- AIC (smaller is better) ---\n")
```

```
##
## --- AIC (smaller is better) ---
```

```
print(round(AIC_mat, 3))
```

```
##      lag Info-Crit
## [1,] 1 2319.905
## [2,] 2 2320.020
## [3,] 3 2321.733
## [4,] 4 2320.204
```

```
cat("\n--- BIC (smaller is better) ---\n")
```

```
##
## --- BIC (smaller is better) ---
```

```
print(round(BIC_mat, 3))
```

```
##      lag Info-Crit
## [1,] 1 2333.382
## [2,] 2 2337.989
## [3,] 3 2344.195
## [4,] 4 2347.158
```

```
# (4) Ljung-Box tests on AR residuals up to 18 lags
cat("PART A : Ljung-Box on AR residuals\n") # FIX: title (was incorrectly "PARTIE B")
```

```
## PART A : Ljung-Box on AR residuals
```

```

LB <- matrix(NA, nrow = max_lag, ncol = pbar)
colnames(LB) <- paste0("AR(", 1:pbar, ")")
rownames(LB) <- paste0("lag", 1:max_lag)

for (p in 1:pbar) {
  fit <- arima(y_t, order = c(p,0,0), include.mean = TRUE)
  res <- residuals(fit)
  for (lag in 1:max_lag) {
    if (lag <= p) {
      LB[lag, p] <- NA
    } else {
      LB[lag, p] <- Box.test(res, lag = lag, type = "Ljung-Box", fitdf = p)$p.value
    }
  }
}

cat("\n--- Ljung-Box p-values on residuals (lags 1..18) ---\n")

```

```

##
## --- Ljung-Box p-values on residuals (lags 1..18) ---

```

```

print(round(LB, 3))

```

```

##      AR(1) AR(2) AR(3) AR(4)
## lag1      NA     NA     NA     NA
## lag2  0.232     NA     NA     NA
## lag3  0.198 0.330     NA     NA
## lag4  0.162 0.242 0.117     NA
## lag5  0.121 0.204 0.118 0.338
## lag6  0.189 0.315 0.216 0.480
## lag7  0.140 0.240 0.181 0.380
## lag8  0.206 0.344 0.282 0.544
## lag9  0.173 0.270 0.213 0.375
## lag10 0.024 0.046 0.035 0.057
## lag11 0.016 0.038 0.029 0.044
## lag12 0.014 0.034 0.026 0.040
## lag13 0.010 0.027 0.021 0.038
## lag14 0.011 0.030 0.025 0.038
## lag15 0.015 0.040 0.033 0.053
## lag16 0.022 0.058 0.049 0.078
## lag17 0.025 0.063 0.054 0.079
## lag18 0.027 0.067 0.055 0.077

```

```

# =====
# PARTIE B : Sélection parmi ARMA(1,1) - ARMA(4,4)
# =====
cat("PARTIE B : ARMA models\n")

```

```

## PARTIE B : ARMA models

```

```

model_labels <- c(
  ARMA11 = "ARMA(1,1)",
  ARMA22 = "ARMA(2,2)",
  ARMA33 = "ARMA(3,3)",
  ARMA44 = "ARMA(4,4)"
)
# -----
# 1) Likelihood Ratio Tests
# Compare nested ARMA(p,p) vs ARMA(p-1,p-1)
# H0: additional AR and MA parameters are jointly zero

cat("\n--- Likelihood Ratio tests (ARMA(p,p) vs ARMA(p-1,p-1)) ---\n")

##
## --- Likelihood Ratio tests (ARMA(p,p) vs ARMA(p-1,p-1)) ---

LR_test <- function(restricted, unrestricted, df) {
  LR <- 2 * (as.numeric(logLik(unrestricted)) - as.numeric(logLik(restricted)))
  # p-value from chi-square distribution
  pval <- pchisq(LR, df = df, lower.tail = FALSE)
  c(LR = LR, df = df, p.value = pval)
}

print(LR_test(models$ARMA11, models$ARMA22, df = 2))

##          LR          df    p.value
## 3.2892250 2.0000000 0.1930874

print(LR_test(models$ARMA22, models$ARMA33, df = 2))

##          LR          df    p.value
## 1.5525307 2.0000000 0.4601212

print(LR_test(models$ARMA33, models$ARMA44, df = 2))

##          LR          df    p.value
## 9.602456703 2.000000000 0.008219644

# 2) Wald Tests
# Test whether the last AR or MA coefficient equals zero
# H0: beta_last = 0
cat("\n--- Wald tests (H0: last AR or last MA coef = 0) ---\n")

##
## --- Wald tests (H0: last AR or last MA coef = 0) ---

# Wald statistic
wald_last_coef <- function(fit, pattern = "^ar") {
  coefs <- fit$coef

```



```

V <- fit$var.coef
# Identify AR or MA coefficients
nm <- grep(pattern, names(coefs), value = TRUE)
if (length(nm) == 0) return(c(W = NA, df = NA, p.value = NA, tested = NA))
# Select the last lag coefficient
last <- nm[length(nm)]
b <- coefs[last]
var_b <- V[last, last]
# Wald statistic
W <- as.numeric(b^2 / var_b)
pval <- pchisq(W, df = 1, lower.tail = FALSE)
c(W = W, df = 1, p.value = pval, tested = last)
}

# Apply Wald test for AR and MA parts separately
cat("\n", model_labels["ARMA22"], "\n"); print(wald_last_coef(models$ARMA22, "^ar")); print(wald_last_coef(models$ARMA22, "^ma"))

##
## ARMA(2,2)

##           W           df           p.value
## "3.16031186221527"      "1" "0.0754487688100612"
##           tested
##           "ar2"

##           W           df           p.value
## "4.53929594662826"      "1" "0.0331252137007518"
##           tested
##           "ma2"

cat("\n", model_labels["ARMA33"], "\n"); print(wald_last_coef(models$ARMA33, "^ar")); print(wald_last_coef(models$ARMA33, "^ma"))

##
## ARMA(3,3)

##           W           df           p.value           tested
## "2.63346958452753"      "1" "0.104632841446918"           "ar3"

##           W           df           p.value           tested
## "1.1170333924321"       "1" "0.290558139171426"           "ma3"

cat("\n", model_labels["ARMA44"], "\n"); print(wald_last_coef(models$ARMA44, "^ar")); print(wald_last_coef(models$ARMA44, "^ma"))

##
## ARMA(4,4)

##           W           df           p.value           tested
## "1.72796391979577"      "1" "0.188671132798028"           "ar4"

##           W           df           p.value           tested
## "0.488224034045135"      "1" "0.484720593500291"           "ma4"

```

```

# 3) Information Criteria (AIC and BIC)
# Select model minimizing information criteria
cat("\n--- Information criteria (ARMA(1,1)..ARMA(4,4)) ---\n")

```

```

##
## --- Information criteria (ARMA(1,1)..ARMA(4,4)) ---

```

```

ARMA_names <- c("ARMA11", "ARMA22", "ARMA33", "ARMA44")
AIC_values <- sapply(models[ARMA_names], AIC)
BIC_values <- sapply(models[ARMA_names], BIC)

print(AIC_values)

```

```

##      ARMA11      ARMA22      ARMA33      ARMA44
## 2320.423 2321.134 2323.582 2317.979

```

```

print(BIC_values)

```

```

##      ARMA11      ARMA22      ARMA33      ARMA44
## 2338.392 2348.088 2359.520 2362.902

```

```

# Identify best models
cat("Best AIC:", model_labels[names(which.min(AIC_values))], "\n")

```

```

## Best AIC: ARMA(4,4)

```

```

cat("Best BIC:", model_labels[names(which.min(BIC_values))], "\n")

```

```

## Best BIC: ARMA(1,1)

```

```

# 4) Ljung-Box Tests on Residuals
# Test for residual autocorrelation up to lag 18
# H0: residuals are white noise
cat("\n--- Ljung-Box residual autocorrelation tests (lags 1..18) ---\n")

```

```

##
## --- Ljung-Box residual autocorrelation tests (lags 1..18) ---

```

```

LB_pvalues <- function(fit, max_lag = 18) {
  u <- residuals(fit)
  p <- length(grep("^ar", names(fit$coef)))
  q <- length(grep("^ma", names(fit$coef)))
  fitdf <- p + q

  sapply((fitdf+1):max_lag, function(L)
    Box.test(u, lag = L, type = "Ljung-Box", fitdf = fitdf)$p.value
  )
}

```

```
LB_ARMA <- lapply(models[ARMA_names], LB_pvalues, max_lag = 18)
for (nm in names(LB_ARMA)) {
  cat("\n", model_labels[nm], "Ljung-Box p-values (lags fitdf+1..18):\n", sep = "")
  print(round(LB_ARMA[[nm]], 3))
}
```

```
##
## ARMA(1,1)Ljung-Box p-values (lags fitdf+1..18):
## [1] 0.230 0.192 0.162 0.260 0.193 0.284 0.228 0.035 0.027 0.025 0.019 0.021
## [13] 0.028 0.042 0.047 0.050
##
## ARMA(2,2)Ljung-Box p-values (lags fitdf+1..18):
## [1] 0.142 0.328 0.294 0.446 0.316 0.057 0.049 0.043 0.041 0.044 0.060 0.087
## [13] 0.090 0.084
##
## ARMA(3,3)Ljung-Box p-values (lags fitdf+1..18):
## [1] 0.094 0.230 0.209 0.018 0.017 0.017 0.017 0.018 0.027 0.043 0.046 0.049
##
## ARMA(4,4)Ljung-Box p-values (lags fitdf+1..18):
## [1] 0.038 0.040 0.066 0.080 0.133 0.185 0.263 0.274 0.213 0.157
```

Interpretation 1 :Les tests de Wald et du ratio de vraisemblance indiquent que l'ajout de retards au-delà de l'ordre 1 n'est pas statistiquement significatif au seuil de 5 %. Les critères d'information AIC et BIC sont minimisés pour le modèle AR(1). Les tests de Ljung-Box montrent toutefois que l'hypothèse de bruit blanc des résidus peut être rejetée pour certains retards relativement élevés (à partir d'environ le 10 retard). Néanmoins, ces autocorrélations apparaissent uniquement à des retards éloignés et demeurent quantitativement limitées. Conformément au principe de parcimonie, et étant donné que les retards supplémentaires ne sont pas significatifs, nous retenons le modèle AR(1), qui fournit une approximation adéquate de la dynamique linéaire de court terme.

Interpretation 2 :Les critères d'information donnent des recommandations différentes : l'AIC favorise ARMA(4,4) tandis que le BIC, plus parcimonieux, favorise ARMA(1,1). Les tests LR ne montrent pas d'amélioration significative en passant de ARMA(1,1) à ARMA(2,2) ni de ARMA(2,2) à ARMA(3,3), alors que l'amélioration vers ARMA(4,4) est significative. Cependant, les tests de Ljung-Box indiquent une autocorrélation résiduelle pour tous les modèles considérés. Conformément au principe de parcimonie, on retient donc le modèle ARMA(1,1).

Question 3 - Réponses dynamiques et prévision

3.a)

Pour les deux modèles sélectionnés, évaluez la réponse dynamique sur un horizon de 10 périodes suivant un choc positif de taille $\sigma = 1,15$ survenant à la première période. Tracez les deux fonctions de réponse dynamique sur le même graphique et commentez. (4 points)

```
library(forecast)

sigma <- 1.15
H <- 10 # Horizon length (10 periods)

# --- Extract coefficients (consistent with forecast::Arima parameterization) ---
phi_ar1 <- as.numeric(AR1$coef["ar1"])

phi_arma <- as.numeric(ARMA11$coef["ar1"])
theta_arma <- as.numeric(ARMA11$coef["ma1"])

# (A) Dynamic response for AR(1)
# IRF_h = sigma * phi^(h-1)
irf_ar1 <- sigma * phi_ar1^(0:(H-1))

# (B) Dynamic response for ARMA(1,1)
# psi0 = 1
# psi1 = phi + theta
# psi_k = phi^(k-2) * (phi + theta), for k >= 3
# IRF = sigma * psi
psi <- numeric(H)

psi[1] <- 1

if (H >= 2) {
  psi[2] <- phi_arma + theta_arma
}

if (H >= 3) {
  for (k in 3:H) {
    psi[k] <- (phi_arma^(k-2)) * (phi_arma + theta_arma)
  }
}

irf_arma11 <- sigma * psi

# Plot both impulse responses on the same graph
steps <- 1:H

library(ggplot2)

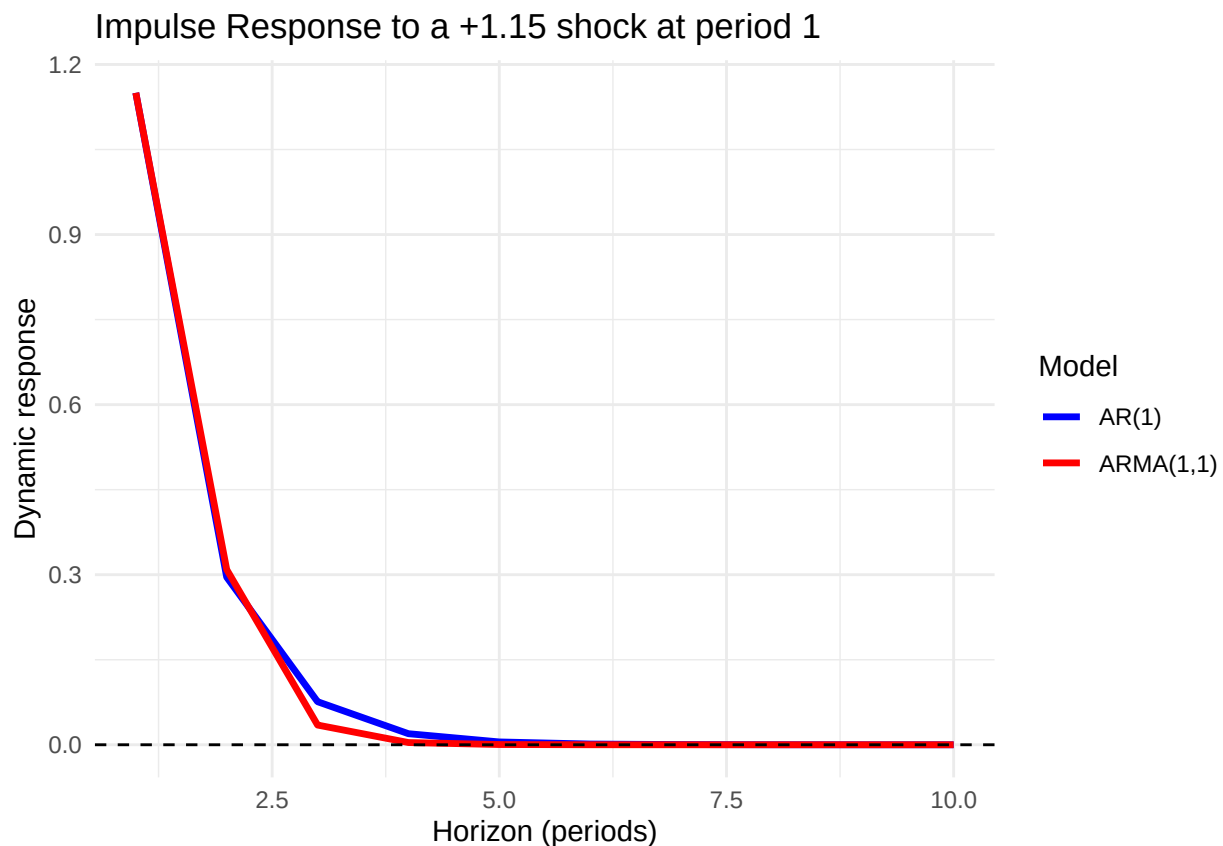
# Ensure numeric format (avoid unexpected object types)
irf_ar1 <- as.numeric(irf_ar1)
irf_arma11 <- as.numeric(irf_arma11)

# Create a long-format data frame for ggplot
```

```
df <- data.frame(
  Step = rep(steps, 2),
  Response = c(irf_ar1, irf_arma11),
  Model = factor(rep(c("AR(1)", "ARMA(1,1)"), each = H))
)

# Generate the plot
ggplot(df, aes(x = Step, y = Response, color = Model)) +
  geom_line(size = 1.2) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    title = paste0("Impulse Response to a +", sigma, " shock at period 1"),
    x = "Horizon (periods)",
    y = "Dynamic response"
  ) +
  theme_minimal() +
  scale_color_manual(values = c("blue", "red"))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



```
# Print numerical values (useful for reporting)
out <- data.frame(
  Step = steps,
  IRF_AR1 = irf_ar1,
  IRF_ARMA11 = irf_arma11
)

print(out)
```

```
##      Step      IRF_AR1      IRF_ARMA11
## 1      1 1.150000e+00 1.150000e+00
## 2      2 2.956188e-01 3.090439e-01
## 3      3 7.599173e-02 3.476262e-02
## 4      4 1.953442e-02 3.910252e-03
## 5      5 5.021515e-03 4.398423e-04
## 6      6 1.290830e-03 4.947539e-05
## 7      7 3.318205e-04 5.565209e-06
## 8      8 8.529773e-05 6.259991e-07
## 9      9 2.192662e-05 7.041513e-08
## 10     10 5.636454e-06 7.920602e-09
```

Interpretation: Les deux fonctions de réponse commencent à 1,15 en période 1, reflétant l'effet immédiat du choc positif. Les réponses décroissent rapidement vers zéro, ce qui indique que l'effet du choc est transitoire et que les processus sont stationnaires. Le modèle ARMA(1,1) présente une légère différence à court terme en raison du terme MA(1), mais les deux modèles convergent rapidement vers l'équilibre.

3.b)

Un choc négatif de taille $\sigma = 1,15$ survient pendant trois périodes consécutives (t , $t+1$, $t+2$). Tracez les fonctions de réponse dynamique des deux modèles sélectionnés sur un horizon de 20 périodes et expliquez ce qui se passe. (4 points)

```
# Question 3.b) Dynamic responses to THREE consecutive shocks
# Shock: negative, size sigma = 1.15, occurs at t, t+1, t+2
# Horizon: 20 periods

library(forecast)
library(ggplot2)

sigma <- 1.15
H <- 20

# ===== Extract coefficients =====
phi_ar1 <- as.numeric(AR1$coef["ar1"])

phi_arma <- as.numeric(ARMA11$coef["ar1"])
theta_arma <- as.numeric(ARMA11$coef["ma1"])

# ===== Helper: impulse response weights psi for AR(1) and ARMA(1,1) =====
psi_ar1 <- function(phi, H){
  # psi_0..psi_{H-1} for AR(1): 1, phi, phi^2, ...
```

```

  phi^(0:(H-1))
}

psi_arma11 <- function(phi, theta, H){
  # psi_0 = 1, psi_1 = phi + theta, psi_h = phi^(h-1)*(phi+theta) for h>=2
  psi <- numeric(H)
  psi[1] <- 1
  if (H >= 2) psi[2] <- phi + theta
  if (H >= 3) {
    for (h in 3:H) {
      psi[h] <- (phi^(h-2)) * (phi + theta)
    }
  }
  psi
}

psi1_ar1 <- psi_ar1(phi_ar1, H)
psi1_arma11 <- psi_arma11(phi_arma, theta_arma, H)

# Three consecutive negative shocks at t, t+1, t+2
# shock sequence e_t: [-sigma, -sigma, -sigma, 0, 0, ...]
e <- c(rep(-sigma, 3), rep(0, H-3))

# Convolution: response[h] = sum_{j=0}^{h-1} psi[j] * e[h-j]
conv_response <- function(psi, e){
  H <- length(e)
  resp <- numeric(H)
  for (h in 1:H){
    s <- 0
    for (j in 1:h){
      # psi[j] corresponds to psi_{j-1}
      # e[h-j+1] corresponds to shock at time (h-j)
      s <- s + psi[j] * e[h-j+1]
    }
    resp[h] <- s
  }
  resp
}

irf3_ar1 <- conv_response(psi1_ar1, e)
irf3_arma11 <- conv_response(psi1_arma11, e)

# Plot both on the same graph
df <- data.frame(
  Step = rep(1:H, 2),
  Response = c(irf3_ar1, irf3_arma11),
  Model = factor(rep(c("AR(1)", "ARMA(1,1)"), each = H))
)

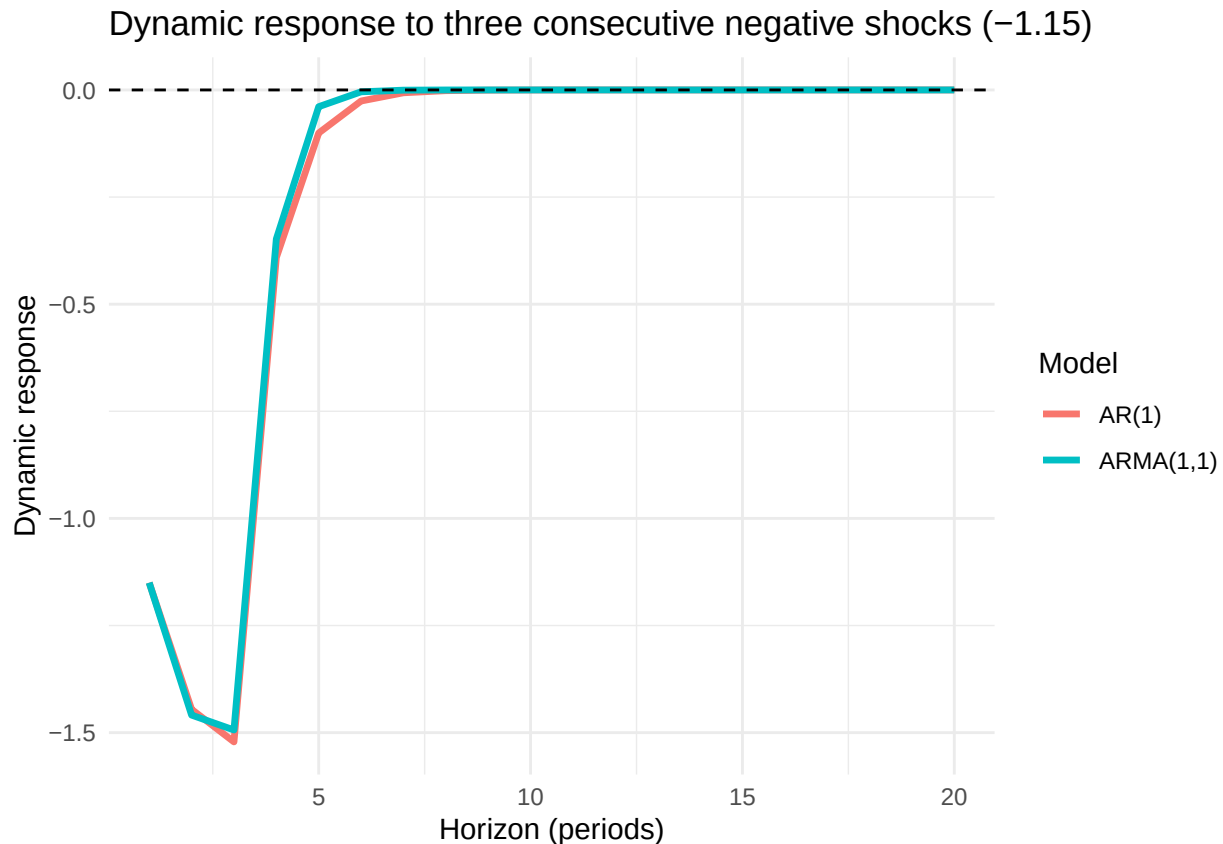
ggplot(df, aes(x = Step, y = Response, color = Model)) +
  geom_line(size = 1.2) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(

```

```

title = paste0("Dynamic response to three consecutive negative shocks (-", sigma, ")"),
x = "Horizon (periods)",
y = "Dynamic response"
) +
theme_minimal()

```



Interpretation: Les trois chocs négatifs successifs produisent un effet cumulatif : la réponse devient de plus en plus négative durant les trois premières périodes, car chaque nouveau choc s'ajoute aux effets retardés des précédents. Après la troisième période, en l'absence de nouveaux chocs, la série revient progressivement vers zéro. Cela confirme que les deux modèles sont stationnaires et que les effets sont transitoires. Les dynamiques de l'AR(1) et de l'ARMA(1,1) sont très proches, avec un ajustement légèrement plus lisse pour le modèle ARMA.

3.c)

Prévision: divisez l'échantillon en un échantillon d'apprentissage et un échantillon de validation. Ce dernier doit contenir les 34 dernières observations. Ré-estimez les deux modèles sélectionnés sur l'échantillon d'apprentissage. Pour chaque modèle:

- Calculez les prévisions à un pas d'avance: $E_t(y_{t+1})$ pour les modèles AR et ARMA sélectionnés.
- Tracez ces prévisions et la vraie série y_t sur un même graphique pour la période de validation.
- Tracez également la vraie série en niveaux z_t avec les prévisions des deux modèles et la prévision naïve: $E_t(z_{t+1}) = z_t$.


```

library(forecast)
library(ggplot2)
#   z_t : level series (numeric vector), length = T
#   y_t : stationary series used for modeling (numeric vector)
#       Typically  $y_t = \text{diff}(z_t) * \text{scale\_factor}$  (NA removed)

n_val <- 34
scale_factor <- 100

# (1) Split into training / validation (last 34 observations in validation)
n <- length(y_t)
idx_val <- (n - n_val + 1):n

y_train <- y_t[-idx_val]
y_val   <- y_t[idx_val]

# (2) Re-estimate the two selected models on TRAINING
fit_AR1   <- forecast::Arima(y_train, order = c(1,0,0), include.mean = TRUE)
fit_ARMA11 <- forecast::Arima(y_train, order = c(1,0,1), include.mean = TRUE)

# (3) 1-step-ahead forecasts over the validation period
#     Idea: keep parameters fixed (estimated on training) and compute fitted values (one-step prediction)
fit_AR1_full   <- forecast::Arima(y_t, model = fit_AR1)
fit_ARMA11_full <- forecast::Arima(y_t, model = fit_ARMA11)

yhat_AR1      <- as.numeric(fitted(fit_AR1_full))[idx_val]
yhat_ARMA11    <- as.numeric(fitted(fit_ARMA11_full))[idx_val]

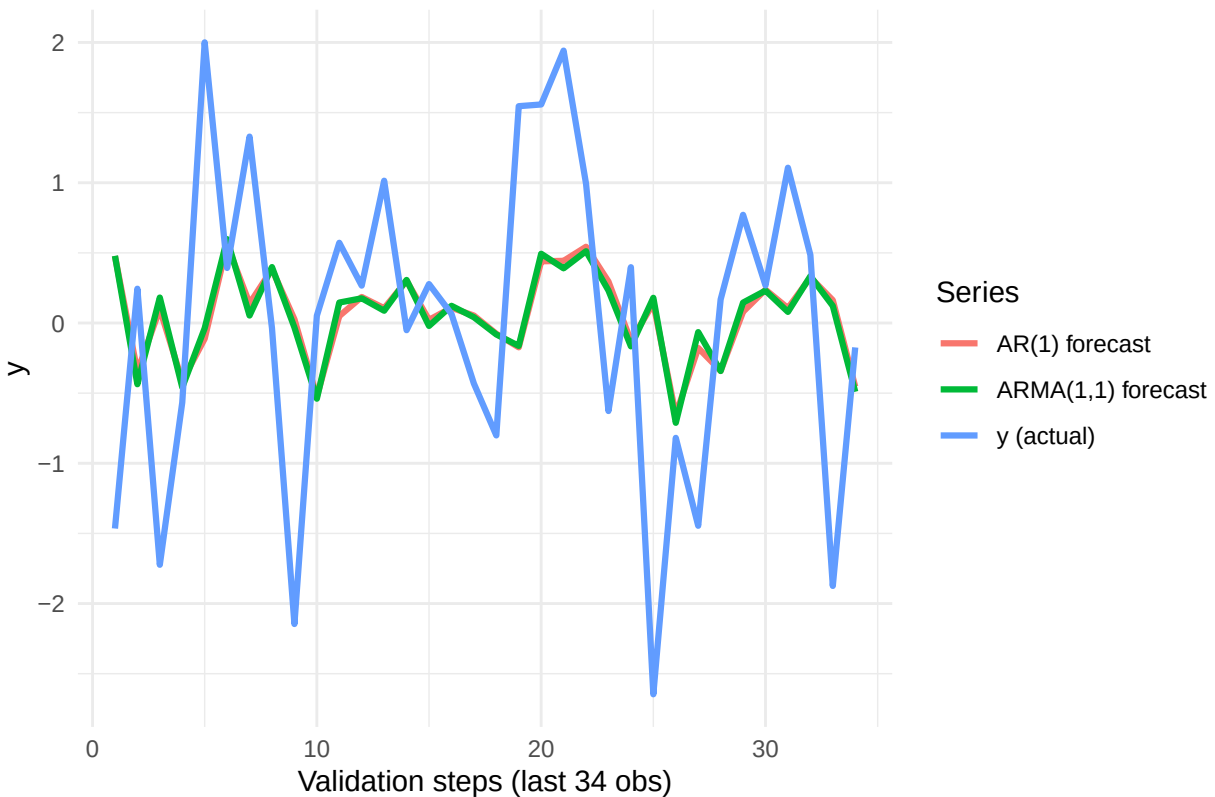
# (4) Plot forecasts vs actual y_t on validation period
steps <- 1:n_val

df_y <- data.frame(
  Step = rep(steps, 3),
  Value = c(as.numeric(y_val), yhat_AR1, yhat_ARMA11),
  Series = factor(rep(c("y (actual)", "AR(1) forecast", "ARMA(1,1) forecast"),
    each = n_val))
)

ggplot(df_y, aes(x = Step, y = Value, color = Series)) +
  geom_line(linewidth = 1.1) +
  labs(title = "Validation period: 1-step-ahead forecasts for y",
    x = "Validation steps (last 34 obs)",
    y = "y") +
  theme_minimal()

```

Validation period: 1-step-ahead forecasts for y



```
# (5) Convert y-forecasts into level forecasts for z
# If  $y_t = \text{diff}(z_t) * \text{scale\_factor}$  (NA removed):
#  $y_t[i]$  corresponds to  $(z_{i+1} - z_i) * \text{scale\_factor}$ 
# So:  $z_{i+1/i} = z_i + \text{yhat}[i] / \text{scale\_factor}$ 
# Align z with y:
#  $z_{\text{prev}}[i] = z_t[i]$  (current level)
#  $z_{\text{next}}[i] = z_t[i+1]$  (next level that  $y_t[i]$  is about)
z_prev <- z_t[1:n]
z_next <- z_t[2:(n+1)]

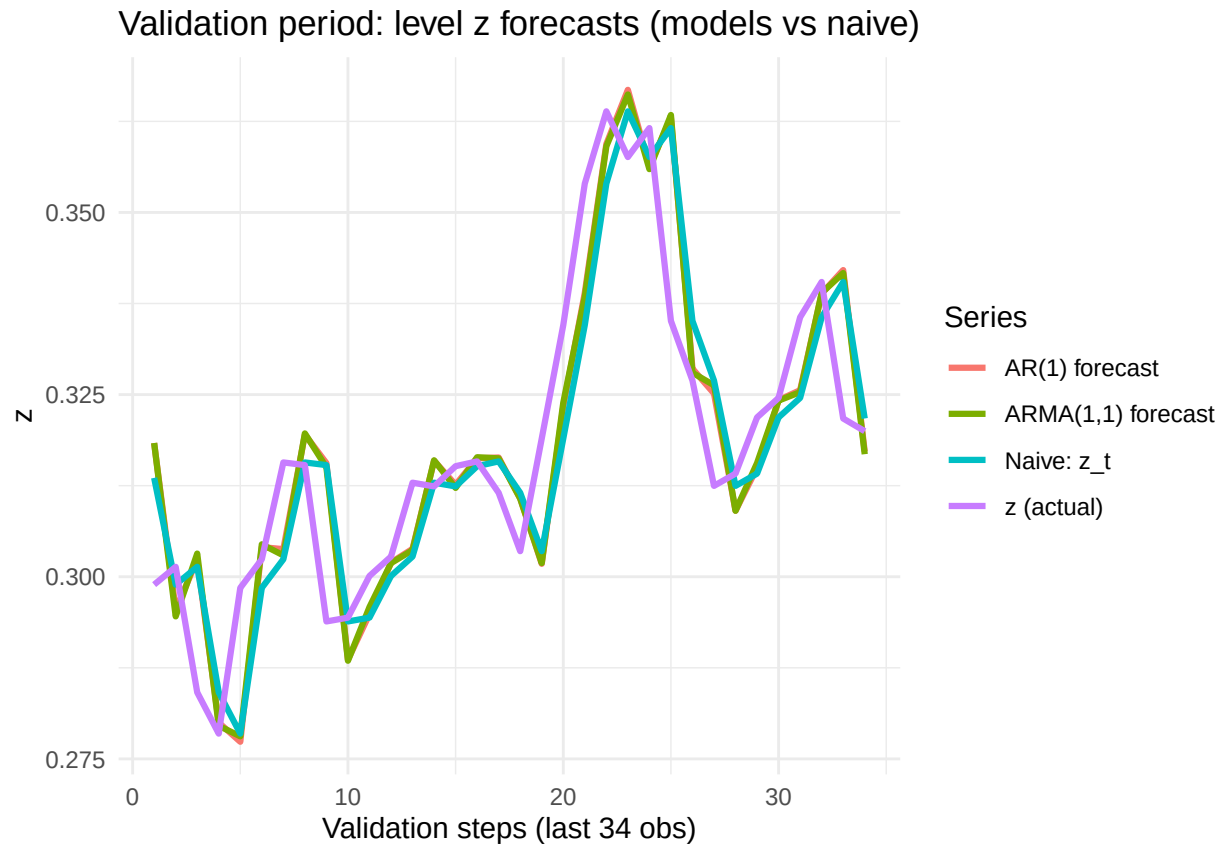
z_true_val <- z_next[idx_val]

z_hat_AR1 <- z_prev[idx_val] + yhat_AR1 / scale_factor
z_hat_ARMA11 <- z_prev[idx_val] + yhat_ARMA11 / scale_factor

# Naive forecast:  $E_t(z_{t+1}) = z_t \Rightarrow$  forecast equals current level
z_hat_naive <- z_prev[idx_val]

# (6) Plot level z: actual vs two model forecasts vs naive
df_z <- data.frame(
  Step = rep(steps, 4),
  Value = c(z_true_val, z_hat_AR1, z_hat_ARMA11, z_hat_naive),
  Series = factor(rep(c("z (actual)", "AR(1) forecast", "ARMA(1,1) forecast", "Naive: z_t"),
    each = n_val))
)
```

```
ggplot(df_z, aes(x = Step, y = Value, color = Series)) +
  geom_line(linewidth = 1.1) +
  labs(title = "Validation period: level z forecasts (models vs naive)",
       x = "Validation steps (last 34 obs)",
       y = "z") +
  theme_minimal()
```



```
# Forecast accuracy measures
# Errors for  $y_t$  forecasts
err_AR1_y <- y_val - yhat_AR1
err_ARMA11_y <- y_val - yhat_ARMA11

rmse <- function(e) sqrt(mean(e^2))
mae <- function(e) mean(abs(e))

cat("Forecast accuracy for y (validation period)\n")
```

```
## Forecast accuracy for y (validation period)
```

```
cat("AR(1) - RMSE:", rmse(err_AR1_y),
    " MAE:", mae(err_AR1_y), "\n")
```

```
## AR(1) - RMSE: 1.13547 MAE: 0.8728431
```

```
cat("ARMA(1,1) - RMSE:", rmse(err_ARMA11_y),
    " MAE:", mae(err_ARMA11_y), "\n")
```

```
## ARMA(1,1) - RMSE: 1.140147 MAE: 0.8780473
```

```
# Errors for z (level forecasts)
err_AR1_z <- z_true_val - z_hat_AR1
err_ARMA11_z <- z_true_val - z_hat_ARMA11
err_naive_z <- z_true_val - z_hat_naive

cat("\nForecast accuracy for z (levels)\n")
```

```
##
## Forecast accuracy for z (levels)
```

```
cat("AR(1) - RMSE:", rmse(err_AR1_z),
    " MAE:", mae(err_AR1_z), "\n")
```

```
## AR(1) - RMSE: 0.0113547 MAE: 0.008728431
```

```
cat("ARMA(1,1) - RMSE:", rmse(err_ARMA11_z),
    " MAE:", mae(err_ARMA11_z), "\n")
```

```
## ARMA(1,1) - RMSE: 0.01140147 MAE: 0.008780473
```

```
cat("Naive - RMSE:", rmse(err_naive_z),
    " MAE:", mae(err_naive_z), "\n")
```

```
## Naive - RMSE: 0.0113703 MAE: 0.00889911
```

Interprétation : Les mesures RMSE et MAE confirment que les modèles AR(1) et ARMA(1,1) présentent des performances très similaires sur l'échantillon de validation. Le modèle AR(1) affiche des erreurs légèrement plus faibles, mais l'écart demeure marginal. De plus, la prévision naïve obtient des résultats comparables en niveaux, ce qui reflète la forte persistance de la série et suggère qu'un modèle plus complexe n'apporte pas d'amélioration substantielle en termes de précision prédictive.

Question 4 - Modéliser la volatilité conditionnelle

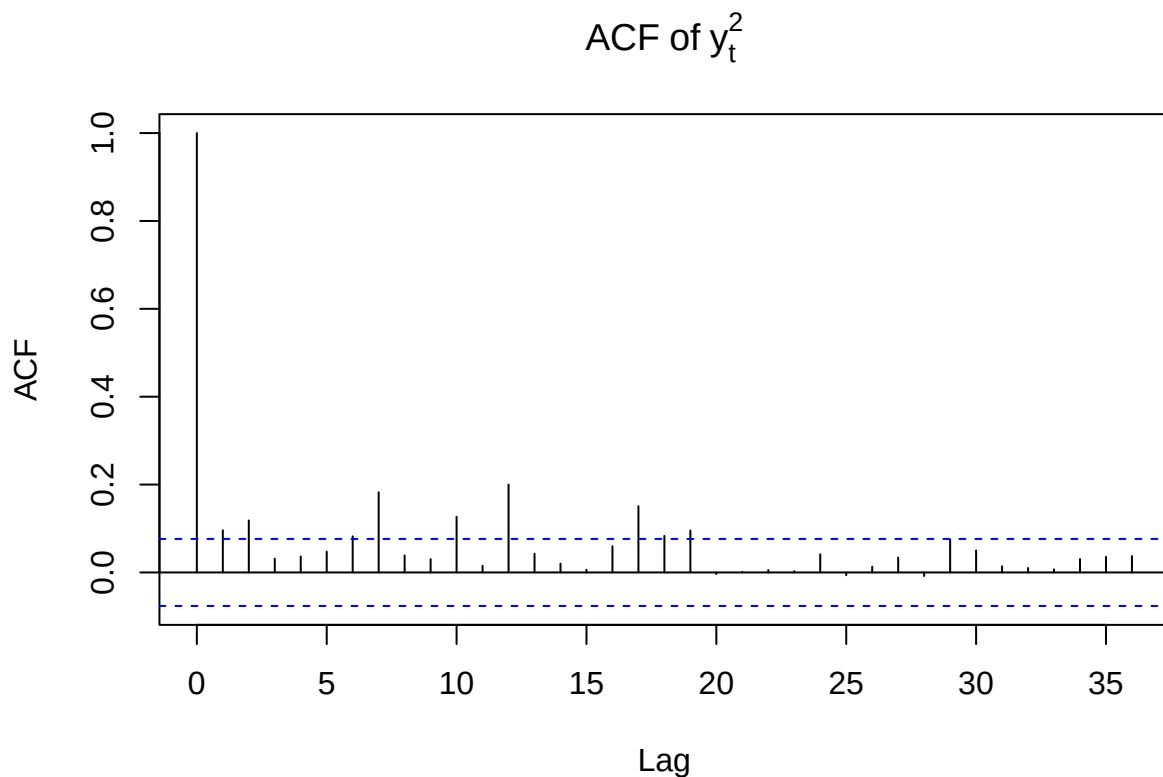
4.a)

Tracez la fonction d'autocorrélation de y_t^2 . Qu'est-ce que vous pouvez conclure?

```
# y_t is your stationary series (already defined as dz)
y_t <- as.numeric(dz)
y_t <- y_t[!is.na(y_t)]

# Compute squared series
y2 <- y_t^2

# Plot ACF of y_t^2 (up to 18 lags as in previous questions)
acf(y2, lag.max = 36,
    main = expression(ACF~"of"~y[t]^2),
    xlab = "Lag",
    ylab = "ACF")
```



Interpretation: L'ACF de y_t^2 montre plusieurs autocorrélations significatives à différents retards (certaines dépassent les bandes de confiance). Cela indique que la variance conditionnelle n'est pas constante dans le temps. On observe donc une dépendance dans les carrés des résidus, ce qui suggère la présence d'effets ARCH et un phénomène de volatilité conditionnelle (volatility clustering). Il est donc pertinent de modéliser la variance à l'aide d'un modèle ARCH ou GARCH.

4.b)

Estimez les versions ARCH(1) et GARCH(1,1) des modèles AR et ARMA sélectionnés à la question 2 à l'aide du maximum de vraisemblance sur l'échantillon d'apprentissage. Présentez les estimations.

```
# Packages
library(rugarch)

## Loading required package: parallel

# (0) Training sample: last 34 obs are validation (as in Q3)
h_out <- 34
n <- length(y_t)
y_train <- y_t[1:(n - h_out)]

# (1) ARCH(1) on AR(1)
#   variance: sGARCH(1,0) ==> ARCH(1)
#   mean      : AR(1)
spec_arch_ar1 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 0)), # ARCH(1)
  mean.model     = list(armaOrder = c(1, 0), include.mean = TRUE), # AR(1)
  distribution.model = "norm"
)

fit_arch_ar1 <- ugarchfit(spec = spec_arch_ar1, data = y_train, solver = "hybrid")

# (2) ARCH(1) on ARMA(1,1)
spec_arch_arma11 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 0)), # ARCH(1)
  mean.model     = list(armaOrder = c(1, 1), include.mean = TRUE), # ARMA(1,1)
  distribution.model = "norm"
)

fit_arch_arma11 <- ugarchfit(spec = spec_arch_arma11, data = y_train, solver = "hybrid")

# (3) GARCH(1,1) on AR(1)
spec_garch_ar1 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)), # GARCH(1,1)
  mean.model     = list(armaOrder = c(1, 0), include.mean = TRUE), # AR(1)
  distribution.model = "norm"
)

fit_garch_ar1 <- ugarchfit(spec = spec_garch_ar1, data = y_train, solver = "hybrid")

# (4) GARCH(1,1) on ARMA(1,1)
spec_garch_arma11 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)), # GARCH(1,1)
  mean.model     = list(armaOrder = c(1, 1), include.mean = TRUE), # ARMA(1,1)
  distribution.model = "norm"
)
```

```
fit_garch_arma11 <- ugarchfit(spec = spec_garch_arma11, data = y_train, solver = "hybrid")

# (5) Output: Maximum Likelihood Estimates
cat("ARCH(1) on AR(1)\n")
```

```
## ARCH(1) on AR(1)
```

```
show(fit_arch_ar1)
```

```
##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,0)
## Mean Model    : ARFIMA(1,0,0)
## Distribution   : norm
##
## Optimal Parameters
## -----
##      Estimate Std. Error  t value Pr(>|t|)
## mu      0.079103   0.072447   1.0919 0.274888
## ar1     0.270501   0.049697   5.4430 0.000000
## omega   1.611804   0.128981  12.4964 0.000000
## alpha1  0.216610   0.072898   2.9714 0.002964
##
## Robust Standard Errors:
##      Estimate Std. Error  t value Pr(>|t|)
## mu      0.079103   0.081351   0.97237 0.330867
## ar1     0.270501   0.066983   4.03838 0.000054
## omega   1.611804   0.361109   4.46348 0.000008
## alpha1  0.216610   0.095047   2.27897 0.022669
##
## LogLikelihood : -1094.92
##
## Information Criteria
## -----
##
## Akaike          3.5109
## Bayes           3.5393
## Shibata         3.5108
## Hannan-Quinn    3.5219
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##                      statistic p-value
## Lag[1]                0.001332 0.9709
## Lag[2*(p+q)+(p+q)-1] [2] 0.455849 0.9739
## Lag[4*(p+q)+(p+q)-1] [5] 3.163172 0.3913
## d.o.f=1
## H0 : No serial correlation
```

```

##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##               statistic p-value
## Lag[1]                0.1348 0.71348
## Lag[2*(p+q)+(p+q)-1][2] 3.9294 0.07877
## Lag[4*(p+q)+(p+q)-1][5] 6.6872 0.06172
## d.o.f=1
##
## Weighted ARCH LM Tests
## -----
##           Statistic Shape Scale P-Value
## ARCH Lag[2]      7.541 0.500 2.000 0.006031
## ARCH Lag[4]      8.075 1.397 1.611 0.015233
## ARCH Lag[6]      9.751 2.222 1.500 0.015846
##
## Nyblom stability test
## -----
## Joint Statistic: 2.5432
## Individual Statistics:
## mu      0.02622
## ar1     0.10122
## omega   1.45875
## alpha1  0.99843
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.07 1.24 1.6
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##           t-value  prob sig
## Sign Bias      0.20574 0.8371
## Negative Sign Bias 0.13184 0.8952
## Positive Sign Bias 0.08017 0.9361
## Joint Effect    0.24886 0.9693
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      24.35      0.1830
## 2    30      34.58      0.2188
## 3    40      50.04      0.1108
## 4    50      54.67      0.2680
##
##
## Elapsed time : 0.101665

```

```
cat("ARCH(1) on ARMA(1,1)\n")
```

```
## ARCH(1) on ARMA(1,1)
```



```
show(fit_arch_arma11)
```

```
##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,0)
## Mean Model    : ARFIMA(1,0,1)
## Distribution   : norm
##
## Optimal Parameters
## -----
##      Estimate  Std. Error  t value Pr(>|t|)
## mu      0.082331    0.070372   1.1699 0.242021
## ar1      0.135214    0.121687   1.1112 0.266497
## ma1      0.166034    0.127507   1.3022 0.192863
## omega    1.591721    0.137164  11.6045 0.000000
## alpha1   0.229466    0.083344   2.7532 0.005901
##
## Robust Standard Errors:
##      Estimate  Std. Error  t value Pr(>|t|)
## mu      0.082331    0.083911   0.98117 0.326507
## ar1      0.135214    0.164769   0.82063 0.411859
## ma1      0.166034    0.152952   1.08553 0.277685
## omega    1.591721    0.348737   4.56424 0.000005
## alpha1   0.229466    0.112104   2.04690 0.040668
##
## LogLikelihood : -1094.18
##
## Information Criteria
## -----
##
## Akaike          3.5118
## Bayes           3.5472
## Shibata         3.5116
## Hannan-Quinn    3.5255
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##                      statistic p-value
## Lag[1]                0.4641  0.4957
## Lag[2*(p+q)+(p+q)-1] [5]  2.6056  0.7197
## Lag[4*(p+q)+(p+q)-1] [9]  4.3245  0.6136
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##                      statistic p-value
## Lag[1]                0.1573  0.69163
```

```

## Lag[2*(p+q)+(p+q)-1][2]      4.2808 0.06338
## Lag[4*(p+q)+(p+q)-1][5]      7.3017 0.04379
## d.o.f=1
##
## Weighted ARCH LM Tests
## -----
##           Statistic Shape Scale P-Value
## ARCH Lag[2]      8.194 0.500 2.000 0.004202
## ARCH Lag[4]      8.818 1.397 1.611 0.009896
## ARCH Lag[6]     10.483 2.222 1.500 0.010503
##
## Nyblom stability test
## -----
## Joint Statistic: 2.8063
## Individual Statistics:
## mu      0.02613
## ar1     0.12129
## ma1     0.12361
## omega   1.48682
## alpha1  0.84097
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.28 1.47 1.88
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##           t-value  prob sig
## Sign Bias      0.02146 0.9829
## Negative Sign Bias 0.25813 0.7964
## Positive Sign Bias 0.21616 0.8289
## Joint Effect    0.27761 0.9642
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      18.60      0.48271
## 2    30      43.87      0.03774
## 3    40      42.50      0.32280
## 4    50      61.54      0.10777
##
##
## Elapsed time : 0.2044089

```

```
cat("GARCH(1,1) on AR(1)\n")
```

```
## GARCH(1,1) on AR(1)
```

```
show(fit_garch_ar1)
```

```

##
## *-----*

```

```

## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(1,0,0)
## Distribution   : norm
##
## Optimal Parameters
## -----
##      Estimate   Std. Error   t value   Pr(>|t|)
## mu      0.066213    0.059372    1.1152   0.264749
## ar1     0.243925    0.040834    5.9736   0.000000
## omega    0.047610    0.034236    1.3907   0.164325
## alpha1   0.151094    0.056876    2.6565   0.007895
## beta1    0.838803    0.061863   13.5591   0.000000
##
## Robust Standard Errors:
##      Estimate   Std. Error   t value   Pr(>|t|)
## mu      0.066213    0.059606    1.11085   0.26663
## ar1     0.243925    0.036029    6.77029   0.00000
## omega    0.047610    0.055641    0.85567   0.39218
## alpha1   0.151094    0.094177    1.60436   0.10863
## beta1    0.838803    0.107519    7.80144   0.00000
##
## LogLikelihood : -1045.662
##
## Information Criteria
## -----
##
## Akaike          3.3567
## Bayes           3.3922
## Shibata         3.3566
## Hannan-Quinn    3.3705
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##
##              statistic p-value
## Lag[1]              1.285  0.2570
## Lag[2*(p+q)+(p+q)-1] [2]    1.581  0.3938
## Lag[4*(p+q)+(p+q)-1] [5]    2.381  0.6005
## d.o.f=1
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##
##              statistic p-value
## Lag[1]              0.1175  0.7318
## Lag[2*(p+q)+(p+q)-1] [5]    1.2612  0.7983
## Lag[4*(p+q)+(p+q)-1] [9]    2.4631  0.8432
## d.o.f=2
##
## Weighted ARCH LM Tests

```

```

## -----
##           Statistic Shape Scale P-Value
## ARCH Lag[3]    0.5155 0.500 2.000 0.4728
## ARCH Lag[5]    0.9315 1.440 1.667 0.7534
## ARCH Lag[7]    1.2681 2.315 1.543 0.8664
##
## Nyblom stability test
## -----
## Joint Statistic: 0.9521
## Individual Statistics:
## mu      0.0595
## ar1     0.1498
## omega   0.6538
## alpha1  0.2711
## beta1   0.4166
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.28 1.47 1.88
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##           t-value   prob sig
## Sign Bias      0.09912 0.9211
## Negative Sign Bias 0.33590 0.7371
## Positive Sign Bias 0.21056 0.8333
## Joint Effect    0.22984 0.9726
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      16.81      0.6026
## 2    30      28.73      0.4793
## 3    40      35.21      0.6433
## 4    50      45.73      0.6067
##
##
## Elapsed time : 0.04673815

```

```
cat("GARCH(1,1) on ARMA(1,1)\n")
```

```
## GARCH(1,1) on ARMA(1,1)
```

```
show(fit_garch_arma11)
```

```

##
## *-----*
## *           GARCH Model Fit           *
## *-----*
##
## Conditional Variance Dynamics
## -----

```

```

## GARCH Model : sGARCH(1,1)
## Mean Model : ARFIMA(1,0,1)
## Distribution : norm
##
## Optimal Parameters
## -----
##      Estimate Std. Error t value Pr(>|t|)
## mu      0.061323  0.056289  1.0894 0.275968
## ar1     -0.015589  0.140304 -0.1111 0.911533
## ma1      0.281086  0.134131  2.0956 0.036117
## omega    0.061907  0.040847  1.5156 0.129623
## alpha1   0.181113  0.064896  2.7908 0.005258
## beta1    0.805733  0.070140 11.4875 0.000000
##
## Robust Standard Errors:
##      Estimate Std. Error t value Pr(>|t|)
## mu      0.061323  0.058469  1.04881 0.294264
## ar1     -0.015589  0.137331 -0.11351 0.909626
## ma1      0.281086  0.128765  2.18294 0.029040
## omega    0.061907  0.061791  1.00187 0.316408
## alpha1   0.181113  0.103533  1.74933 0.080235
## beta1    0.805733  0.113017  7.12933 0.000000
##
## LogLikelihood : -1043.919
##
## Information Criteria
## -----
##
## Akaike      3.3544
## Bayes       3.3969
## Shibata     3.3542
## Hannan-Quinn 3.3709
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##                      statistic p-value
## Lag[1]                0.4436  0.5054
## Lag[2*(p+q)+(p+q)-1] [5]  1.5094  0.9978
## Lag[4*(p+q)+(p+q)-1] [9]  2.1517  0.9772
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##                      statistic p-value
## Lag[1]                0.2633  0.6079
## Lag[2*(p+q)+(p+q)-1] [5]  1.5229  0.7343
## Lag[4*(p+q)+(p+q)-1] [9]  2.9187  0.7723
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##      Statistic Shape Scale P-Value
## ARCH Lag[3]    0.4928 0.500 2.000  0.4827

```

```

## ARCH Lag[5]      0.9574 1.440 1.667 0.7456
## ARCH Lag[7]      1.4279 2.315 1.543 0.8357
##
## Nyblom stability test
## -----
## Joint Statistic: 1.1574
## Individual Statistics:
## mu      0.06277
## ar1     0.22348
## ma1     0.24835
## omega   0.75945
## alpha1  0.31342
## beta1   0.46417
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.49 1.68 2.12
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##              t-value  prob sig
## Sign Bias      0.2565 0.7977
## Negative Sign Bias 0.3636 0.7163
## Positive Sign Bias 0.5364 0.5919
## Joint Effect    0.5024 0.9184
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      11.19      0.9173
## 2    30      22.69      0.7905
## 3    40      35.73      0.6200
## 4    50      57.55      0.1883
##
##
## Elapsed time : 0.05969691

```

Interpretation of Estimated Models

AR(1)–ARCH(1)

Mean equation:

$$y_t = -0.079103 + 0.270501y_{t-1} + u_t$$

Variance equation:

$$h_t = 1.611804 + 0.216610u_{t-1}^2$$

ARMA(1,1)–ARCH(1)

Mean equation:

$$y_t = 0.082331 + 0.135214y_{t-1} + u_t - 0.166034u_{t-1}$$

Variance equation:

$$h_t = 1.591721 + 0.229466u_{t-1}^2$$

AR(1)–GARCH(1,1)

Mean equation:

$$y_t = 0.066213 + 0.243925y_{t-1} + u_t$$

Variance equation:

$$h_t = 0.047610 + 0.151094u_{t-1}^2 + 0.838803h_{t-1}$$

ARMA(1,1)–GARCH(1,1)

Mean equation:

$$y_t = 0.061323 - 0.015589y_{t-1} + u_t - 0.281086u_{t-1}$$

Variance equation:

$$h_t = 0.061907 + 0.181113u_{t-1}^2 + 0.805733h_{t-1}$$

Interprétation : Les paramètres estimés satisfont les conditions usuelles d'un modèle GARCH(1,1), à savoir $\omega > 0$, $\alpha_1 \geq 0$ et $\beta_1 \geq 0$. De plus, la condition de stationnarité de la variance conditionnelle est vérifiée puisque $\alpha_1 + \beta_1 < 1$. Cela garantit l'existence d'une variance inconditionnelle finie et confirme que le processus est stationnaire au sens de la variance.

4.c)

Effectuez des tests de rapport de vraisemblance pour choisir entre AR, AR-ARCH(1) et AR-GARCH(1,1). Répétez la procédure pour ARMA, ARMA-ARCH(1) et ARMA-GARCH(1,1).

```
# Likelihood Ratio Tests:
# (1) AR vs AR-ARCH(1) vs AR-GARCH(1,1)
# (2) ARMA vs ARMA-ARCH(1) vs ARMA-GARCH(1,1)
# Using MLE on training sample

# 0) Training / validation split (last 34 obs = validation)
h_out <- 34
n <- length(y_t)
y_train <- y_t[1:(n - h_out)]
y_valid <- y_t[(n - h_out + 1):n]

# 1) Mean models (Gaussian MLE) on training sample
AR1_train <- forecast::Arima(y_train, order = c(1,0,0), include.mean = TRUE, method = "ML")
```

```

ARMA11_train <- forecast::Arima(y_train, order = c(1,0,1), include.mean = TRUE, method = "ML")

# Helper: log-likelihood and number of parameters for Arima()
ll_arima <- function(fit) as.numeric(logLik(fit))
k_arima <- function(fit) as.numeric(attr(logLik(fit), "df")) # includes sigma^2

# 2) ARCH/GARCH specs (rugarch) on training sample
# --- ARCH(1) on AR(1) ---
spec_arch_ar1 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 0)), # ARCH(1)
  mean.model = list(armaOrder = c(1, 0), include.mean = TRUE),
  distribution.model = "norm"
)
fit_arch_ar1 <- ugarchfit(spec = spec_arch_ar1, data = y_train, solver = "hybrid")

# --- GARCH(1,1) on AR(1) ---
spec_garch_ar1 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)), # GARCH(1,1)
  mean.model = list(armaOrder = c(1, 0), include.mean = TRUE),
  distribution.model = "norm"
)
fit_garch_ar1 <- ugarchfit(spec = spec_garch_ar1, data = y_train, solver = "hybrid")

# --- ARCH(1) on ARMA(1,1) ---
spec_arch_arma11 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 0)), # ARCH(1)
  mean.model = list(armaOrder = c(1, 1), include.mean = TRUE),
  distribution.model = "norm"
)
fit_arch_arma11 <- ugarchfit(spec = spec_arch_arma11, data = y_train, solver = "hybrid")

# --- GARCH(1,1) on ARMA(1,1) ---
spec_garch_arma11 <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)), # GARCH(1,1)
  mean.model = list(armaOrder = c(1, 1), include.mean = TRUE),
  distribution.model = "norm"
)
fit_garch_arma11 <- ugarchfit(spec = spec_garch_arma11, data = y_train, solver = "hybrid")

# Helper: log-likelihood and number of parameters for ugarchfit
ll_ugarch <- function(fit) as.numeric(fit@fit$LLH)
k_ugarch <- function(fit) length(coef(fit))

# 3) Likelihood Ratio test function
lr_test <- function(ll0, k0, ll1, k1, name0 = "Model0", name1 = "Model1") {
  LR <- 2 * (ll1 - ll0)
  df <- k1 - k0
  p <- 1 - pchisq(LR, df = df)
  data.frame(Comparison = paste(name0, "vs", name1), LR = LR, df = df, p_value = p)
}

```


4) AR family: AR vs AR-ARCH vs AR-GARCH

```
ll_ar      <- ll_arma(AR1_train);    k_ar      <- k_arma(AR1_train)
ll_arch    <- ll_ugarch(fit_arch_ar1); k_arch    <- k_ugarch(fit_arch_ar1)
ll_garch    <- ll_ugarch(fit_garch_ar1); k_garch    <- k_ugarch(fit_garch_ar1)

LR_AR_1    <- lr_test(ll_ar,    k_ar,    ll_arch,    k_arch,    "AR(1)", "AR(1)-ARCH(1)")
LR_AR_2    <- lr_test(ll_ar,    k_ar,    ll_garch,    k_garch,    "AR(1)", "AR(1)-GARCH(1,1)")
LR_AR_3    <- lr_test(ll_arch, k_arch, ll_garch, k_garch, "AR(1)-ARCH(1)", "AR(1)-GARCH(1,1)")
```

5) ARMA family: ARMA vs ARMA-ARCH vs ARMA-GARCH

```
ll_arma     <- ll_arma(ARMA11_train);    k_arma     <- k_arma(ARMA11_train)
ll_archA    <- ll_ugarch(fit_arch_arma11); k_archA    <- k_ugarch(fit_arch_arma11)
ll_garchA    <- ll_ugarch(fit_garch_arma11); k_garchA    <- k_ugarch(fit_garch_arma11)

LR_ARMA_1   <- lr_test(ll_arma, k_arma, ll_archA, k_archA, "ARMA(1,1)", "ARMA(1,1)-ARCH(1)")
LR_ARMA_2   <- lr_test(ll_arma, k_arma, ll_garchA, k_garchA, "ARMA(1,1)", "ARMA(1,1)-GARCH(1,1)")
LR_ARMA_3   <- lr_test(ll_archA, k_archA, ll_garchA, k_garchA, "ARMA(1,1)-ARCH(1)", "ARMA(1,1)-GARCH(1,1)")
```

6) Print results

```
LR_results <- rbind(LR_AR_1, LR_AR_2, LR_AR_3, LR_ARMA_1, LR_ARMA_2, LR_ARMA_3)
print(LR_results)
```

##	Comparison	LR	df	p_value
## 1	AR(1) vs AR(1)-ARCH(1)	16.29783	1	5.412588e-05
## 2	AR(1) vs AR(1)-GARCH(1,1)	114.81484	2	0.000000e+00
## 3	AR(1)-ARCH(1) vs AR(1)-GARCH(1,1)	98.51701	1	0.000000e+00
## 4	ARMA(1,1) vs ARMA(1,1)-ARCH(1)	16.15048	1	5.850389e-05
## 5	ARMA(1,1) vs ARMA(1,1)-GARCH(1,1)	116.67193	2	0.000000e+00
## 6	ARMA(1,1)-ARCH(1) vs ARMA(1,1)-GARCH(1,1)	100.52145	1	0.000000e+00

Interpretation: # Dans les deux familles (AR et ARMA), le modèle GARCH(1,1) domine significativement les versions ARCH(1) et homoscédastiques. Le modèle final recommandé est donc : ARMA(1,1)-GARCH(1,1) car il offre la meilleure vraisemblance et capture correctement la dynamique de volatilité.

4.d)

Effectuez un test de spécification de la volatilité conditionnelle sur le AR-ARCH(1) et le AR-GARCH(1,1). Quel est le meilleur modèle pour la volatilité conditionnelle?

```
max_lag <- 18
```

*# 1) Get standardized residuals: $z_t = u_t / \sqrt{h_t}$
 # If volatility model is well specified, z_t should be $\sim i.i.d.$
 # Then z_t^2 should have no autocorrelation (no remaining ARCH effects).*

```
z_arch <- residuals(fit_arch_ar1, standardize = TRUE)
z_garch <- residuals(fit_garch_ar1, standardize = TRUE)

z2_arch <- as.numeric(z_arch)^2
```

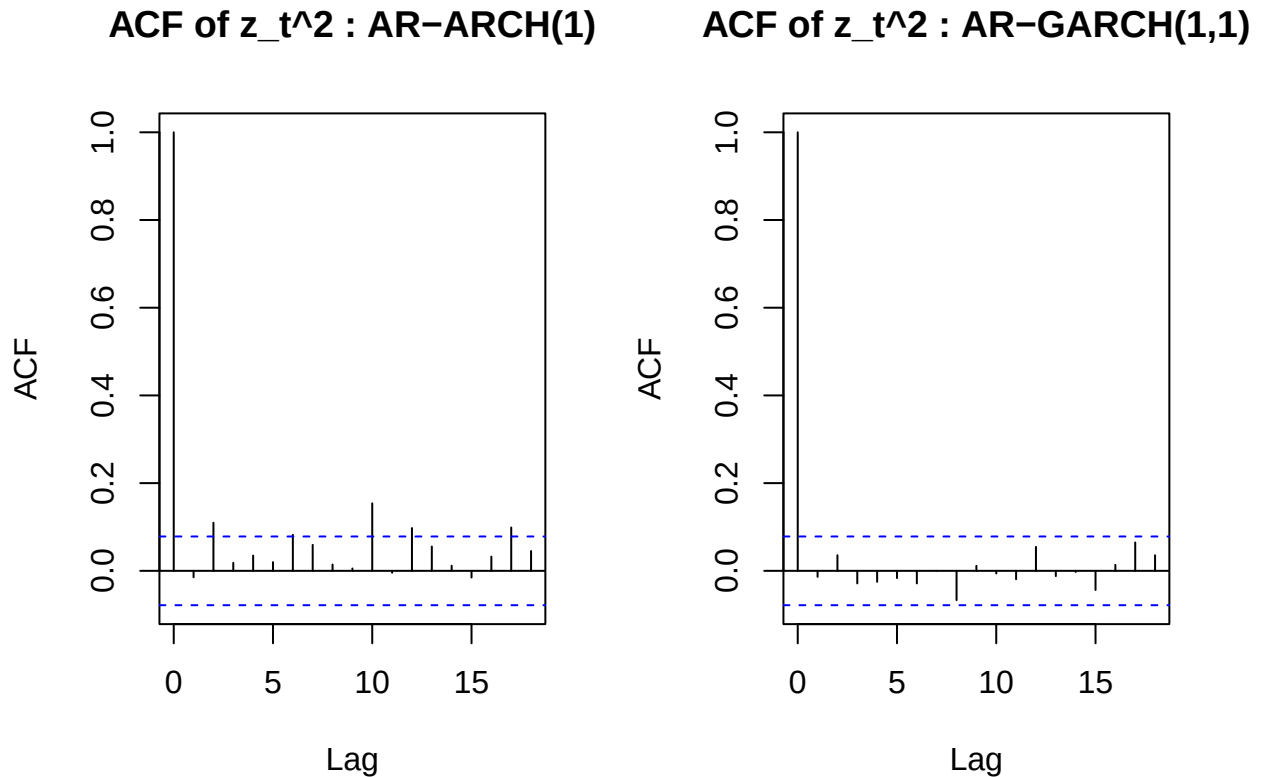
```
z2_garch <- as.numeric(z_garch)^2
```

```
# 2) ACF of squared standardized residuals
```

```
par(mfrow = c(1,2))
```

```
acf(z2_arch, lag.max = max_lag, main = "ACF of z_t^2 : AR-ARCH(1)")
```

```
acf(z2_garch, lag.max = max_lag, main = "ACF of z_t^2 : AR-GARCH(1,1)")
```



```
par(mfrow = c(1,1))
```

```
# 3) Ljung-Box tests on squared standardized residuals
```

```
# H0: no serial correlation in z_t^2 (=> no remaining ARCH effects)
```

```
LB_arch <- sapply(1:max_lag, function(L) Box.test(z2_arch, lag = L, type = "Ljung-Box")$p.value)
```

```
LB_garch <- sapply(1:max_lag, function(L) Box.test(z2_garch, lag = L, type = "Ljung-Box")$p.value)
```

```
results_spec <- data.frame(
  Lag = 1:max_lag,
  pval_ARCH = round(LB_arch, 4),
  rej_ARCH = as.integer(LB_arch < 0.05),
  pval_GARCH = round(LB_garch, 4),
  rej_GARCH = as.integer(LB_garch < 0.05)
)
```

```
print(results_spec)
```

```
##      Lag pval_ARCH rej_ARCH pval_GARCH rej_GARCH
## 1      1   0.7135      0   0.7318      0
## 2      2   0.0210      1   0.6325      0
## 3      3   0.0474      1   0.6972      0
## 4      4   0.0691      0   0.7665      0
## 5      5   0.1112      0   0.8486      0
## 6      6   0.0395      1   0.8655      0
## 7      7   0.0305      1   0.9251      0
## 8      8   0.0486      1   0.7182      0
## 9      9   0.0754      0   0.7939      0
## 10     10   0.0006      1   0.8578      0
## 11     11   0.0012      1   0.8925      0
## 12     12   0.0002      1   0.8141      0
## 13     13   0.0002      1   0.8618      0
## 14     14   0.0004      1   0.9036      0
## 15     15   0.0006      1   0.8804      0
## 16     16   0.0008      1   0.9106      0
## 17     17   0.0002      1   0.8139      0
## 18     18   0.0002      1   0.8155      0
```

```
cat("\nNumber of rejections (5%) up to lag", max_lag, ":\n")
```

```
##
## Number of rejections (5%) up to lag 18 :
```

```
cat("AR-ARCH(1)  :", sum(LB_arch < 0.05), "\n")
```

```
## AR-ARCH(1)  : 14
```

```
cat("AR-GARCH(1,1):", sum(LB_garch < 0.05), "\n")
```

```
## AR-GARCH(1,1): 0
```

Interpretation: Les tests de Ljung-Box sont effectués sur les résidus standardisés au carré afin de détecter des effets ARCH résiduels. AR-ARCH(1): Sur 18 retards, 14 rejets au seuil de 5%. Les résidus au carré présentent encore une autocorrélation significative. Le modèle ARCH(1) ne capture pas complètement la dynamique de la volatilité conditionnelle. AR-GARCH(1,1): Aucun rejet au seuil de 5%. Les résidus au carré ne présentent pas d'autocorrélation significative. Donc, Le modèle GARCH(1,1) capture correctement l'hétéroscédasticité conditionnelle.

4.e)

Comme dans la partie 3, tracez les prévisions à un pas pour y_t pour la période couverte par l'échantillon de validation. Comparez-les à celles obtenues dans la partie 3.

```

###      Compare Part 3 (AR1/ARMA11) vs Part 4 (AR/ARMA + ARCH/GARCH)
library(tidyrr)

n      <- length(y_t)

y_train <- y_t[1:(n - h_out)]
y_test  <- y_t[(n - h_out + 1):n]

# (1) Rolling forecasts for AR/ARMA (Part 3)
#      Expanding window: re-estimate each time, 1-step ahead
rolling_arima_forecast <- function(y_full, h_out, order) {
  n <- length(y_full)
  fc <- rep(NA, h_out)

  for (i in 1:h_out) {
    y_sub <- y_full[1:(n - h_out + i - 1)] # expanding window
    fit   <- forecast::Arima(y_sub, order = order, include.mean = TRUE, method = "ML")
    fc[i] <- as.numeric(forecast::forecast(fit, h = 1)$mean)
  }
  fc
}

f_ar1_roll    <- rolling_arima_forecast(y_t, h_out, order = c(1,0,0))
f_arma11_roll <- rolling_arima_forecast(y_t, h_out, order = c(1,0,1))

# (2) Rolling forecasts for AR/ARMA + ARCH(1) / GARCH(1,1) (Part 4)
#      Note: ARCH(1) = sGARCH(1,0) in rugarch (garchOrder = c(1,0))
rolling_garch_forecast <- function(y_full, h_out, arma_order, garch_order) {
  n <- length(y_full)
  fc <- rep(NA, h_out)

  for (i in 1:h_out) {
    y_sub <- y_full[1:(n - h_out + i - 1)] # expanding window

    spec <- ugarchspec(
      mean.model      = list(armaOrder = arma_order, include.mean = TRUE),
      variance.model  = list(model = "sGARCH", garchOrder = garch_order),
      distribution.model = "norm"
    )

    fit <- tryCatch(
      ugarchfit(spec = spec, data = y_sub, solver = "hybrid"),
      error = function(e) NULL
    )

    if (!is.null(fit)) {
      fc1 <- ugarchforecast(fit, n.ahead = 1)
      fc[i] <- as.numeric(fitted(fc1)) # point forecast of  $y_{t+1|t}$ 
    } else {
      fc[i] <- NA
    }
  }
}

```

```

    fc
  }

# --- ARCH(1) ---
f_ar1_arch_roll    <- rolling_garch_forecast(y_t, h_out, arma_order = c(1,0), garch_order = c(1,0))
f_arma11_arch_roll <- rolling_garch_forecast(y_t, h_out, arma_order = c(1,1), garch_order = c(1,0))

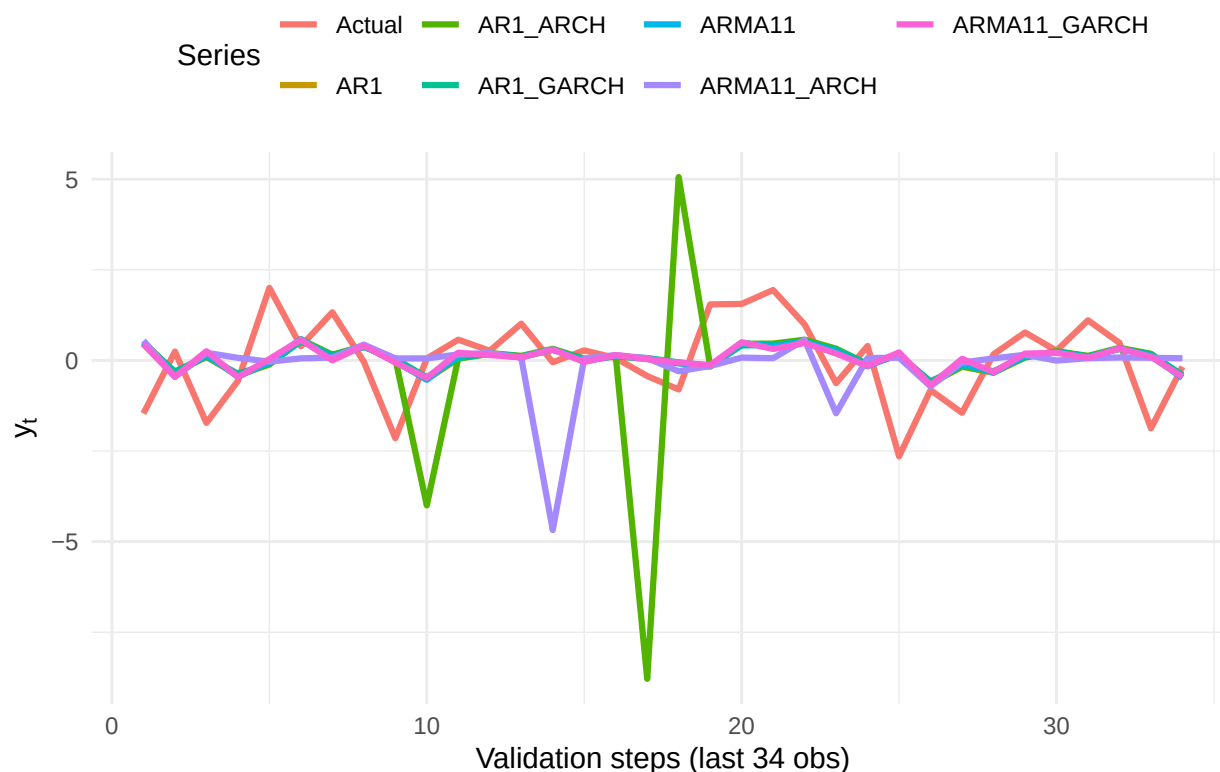
# --- GARCH(1,1) ---
f_ar1_garch_roll    <- rolling_garch_forecast(y_t, h_out, arma_order = c(1,0), garch_order = c(1,1))
f_arma11_garch_roll <- rolling_garch_forecast(y_t, h_out, arma_order = c(1,1), garch_order = c(1,1))

# (3) Plot: Actual vs all rolling 1-step forecasts
plot_df <- data.frame(
  t = 1:h_out,
  Actual      = y_test,
  AR1         = f_ar1_roll,
  ARMA11      = f_arma11_roll,
  AR1_ARCH    = f_ar1_arch_roll,
  AR1_GARCH   = f_ar1_garch_roll,
  ARMA11_ARCH = f_arma11_arch_roll,
  ARMA11_GARCH = f_arma11_garch_roll
) %>%
  pivot_longer(-t, names_to = "Series", values_to = "y")

ggplot(plot_df, aes(x = t, y = y, color = Series)) +
  geom_line(linewidth = 1.1) +
  theme_minimal() +
  labs(
    title = "Rolling 1-step ahead forecasts vs Actual (Part 3 vs Part 4)",
    x = "Validation steps (last 34 obs)",
    y = expression(y[t])
  ) +
  theme(legend.position = "top")

```

Rolling 1-step ahead forecasts vs Actual (Part 3 vs Part 4)



Interpretation: Les prévisions à un pas des modèles AR(1), ARMA(1,1) et de leurs versions ARCH/GARCH sont très similaires sur l'échantillon de validation. L'ajout d'ARCH ou de GARCH n'améliore pas significativement les prévisions ponctuelles de la moyenne. Les différences concernent surtout la variance conditionnelle. Ainsi, les modèles GARCH apportent une amélioration pour la modélisation de la volatilité, mais pas pour la prévision de la moyenne à court terme.

4.f)

Comparez l'erreur quadratique moyenne de la prévision à un pas pour y_t pour les deux modèles sélectionnés dans la partie 3, les deux nouveaux modèles, la prévision naïve $E_t y_{t+1} = y_t$ et la prévision paresseuse $E_t y_{t+1} = 0$ pour tout t . La modélisation de la covariance conditionnelle était-elle utile ?

```
### Models:
###   Benchmarks : Lazy (0), Naive (y_t)
###   Part 3      : AR(1), ARMA(1,1)
###   Part 4      : AR(1)-GARCH(1,1), ARMA(1,1)-GARCH(1,1)
# Setup
h_out  <- 34
n      <- length(y_t)
y_valid <- y_t[(n - h_out + 1):n]

# Rolling 1-step forecasts: AR / ARMA (Part 3)
rolling_arma_1step <- function(y_full, h_out, order) {
  n      <- length(y_full)
  preds <- numeric(h_out)
```

```

for (i in 1:h_out) {
  y_sub <- y_full[1:(n - h_out + i - 1)] # data up to time t
  fit <- forecast::Arima(
    y_sub,
    order = order,
    include.mean = TRUE,
    method = "ML"
  )
  preds[i] <- as.numeric(forecast::forecast(fit, h = 1)$mean)
}

return(preds)
}

fc_ar1_roll <- rolling_arima_1step(y_t, h_out, c(1, 0, 0))
fc_arma11_roll <- rolling_arima_1step(y_t, h_out, c(1, 0, 1))

# Rolling 1-step forecasts: AR/ARMA + GARCH(1,1) (Part 4)
rolling_garch_1step <- function(y_full, h_out, arma_order, garch_order = c(1,1)) {
  n <- length(y_full)
  preds <- rep(NA_real_, h_out)

  for (i in 1:h_out) {

    y_sub <- y_full[1:(n - h_out + i - 1)]

    spec <- ugarchspec(
      mean.model = list(
        armaOrder = arma_order,
        include.mean = TRUE
      ),
      variance.model = list(
        model = "sGARCH",
        garchOrder = garch_order
      ),
      distribution.model = "norm"
    )

    fit <- tryCatch(
      ugarchfit(spec, data = y_sub, solver = "hybrid"),
      error = function(e) NULL
    )

    if (!is.null(fit)) {
      fc1 <- ugarchforecast(fit, n.ahead = 1)
      preds[i] <- as.numeric(fitted(fc1))
    }
  }

  return(preds)
}

fc_ar1_garch_roll <- rolling_garch_1step(y_t, h_out, arma_order = c(1, 0))

```

```
fc_arma11_garch_roll <- rolling_garch_1step(y_t, h_out, arma_order = c(1, 1))
```

```
# Benchmarks
```

```
# Naive:  $E_t[y_{t+1}] = y_t$ 
```

```
fc_naive <- y_t[(n - h_out):(n - 1)]
```

```
# Lazy: always predict 0
```

```
fc_lazy <- rep(0, h_out)
```

```
# RMSE function
```

```
rmse <- function(actual, pred) {  
  sqrt(mean((actual - pred)^2, na.rm = TRUE))  
}
```

```
# Compute RMSE for all models
```

```
rmse_table <- data.frame(  
  Model = c(  
    "Lazy :  $E[y_{t+1}] = 0$ ",  
    "Naive :  $E[y_{t+1}] = y_t$ ",  
    "AR(1) [Part 3]",  
    "ARMA(1,1) [Part 3]",  
    "AR(1)-GARCH(1,1) [Part 4]",  
    "ARMA(1,1)-GARCH(1,1) [Part 4]"  
  ),  
  RMSE = c(  
    rmse(y_valid, fc_lazy),  
    rmse(y_valid, fc_naive),  
    rmse(y_valid, fc_ar1_roll),  
    rmse(y_valid, fc_arma11_roll),  
    rmse(y_valid, fc_ar1_garch_roll),  
    rmse(y_valid, fc_arma11_garch_roll)  
  )  
)
```

```
# Sort best to worst
```

```
rmse_table <- rmse_table[order(rmse_table$RMSE), ]  
rownames(rmse_table) <- NULL
```

```
cat(" RMSE - One-step Rolling Forecasts (Validation)\n")
```

```
## RMSE - One-step Rolling Forecasts (Validation)
```

```
print(rmse_table)
```

```
##           Model      RMSE  
## 1 AR(1)-GARCH(1,1) [Part 4] 1.134622  
## 2 AR(1) [Part 3] 1.136296  
## 3 Lazy :  $E[y_{t+1}] = 0$  1.137030  
## 4 ARMA(1,1) [Part 3] 1.141381  
## 5 ARMA(1,1)-GARCH(1,1) [Part 4] 1.143343  
## 6 Naive :  $E[y_{t+1}] = y_t$  1.513434
```


Interpretation: Le modèle AR(1)-GARCH(1,1) présente la plus faible RMSE, donc la meilleure performance en prévision à un pas sur l'échantillon de validation. Cependant, l'amélioration par rapport au modèle AR(1) est très faible (1.1346 contre 1.1363). Cela indique que la modélisation de la variance conditionnelle (GARCH) n'améliore que très marginalement la prévision de la moyenne. Le modèle naïf ($\hat{y}_{t+1} = y_t$) est clairement moins performant, tandis que la prévision paresseuse (0) donne un résultat proche des modèles AR, ce qui suggère que la série est centrée autour de zéro. En conclusion, la modélisation de l'hétéroscédasticité conditionnelle est surtout utile pour prévoir la volatilité, mais apporte peu d'amélioration pour la prévision du niveau moyen à court terme.